Overview of
“**Building A Large Language Model**”

Foundations of LLMs & Architecture

- **Focus:** Understanding how models process language and the internal mechanics of the Transformer.
- Data Processing & Embeddings (Unit I)
  - **Tokenization:** Converting raw text into IDs using **Byte Pair Encoding (BPE)** and adding special context tokens.
  - **Embeddings:** Representing words as high-dimensional vectors and encoding **positional information** so the model understands word order.
  - **Data Sampling:** Using a **sliding window** approach to create manageable training batches from massive datasets.
- The Transformer Core (Unit II)
  - **The Attention Mechanism**: Solving long-sequence dependency issues by "attending" to different parts of an input simultaneously
  - **Causal & Multi-Head Attention**: Implementing **causal masking** (hiding future words) and parallelizing attention via multiple "heads”.
  - **Building the GPT Block**: Integrating Layer Normalization, GELU activations, and Shortcut Connections to stabilize deep network training.

Training, Fine-Tuning & Application

- **Focus:** Moving from raw architecture to a functional, task-specific AI assistant.
- Pretraining & Generative Logic (Unit III)
  - **Unlabeled Data**: Training the model to predict the next token on vast, raw datasets to build general world knowledge.
  - **Decoding Strategies**: Implementing techniques like temperature scaling and top-k sampling to control the randomness and creativity of generated text.
  - **Weight Management**: Practical skills in loading/saving PyTorch weights and utilizing OpenAI's pretrained models.
- Fine-Tuning for Specific Tasks (Units III & IV)
  - **Classification Head**: Adapting a pretrained LLM for discriminative tasks (e.g., a Spam Classifier) by adding specialized output layers.
  - **Instruction Fine-Tuning**: The transition from a "completion" engine to an "assistant" by training on supervised (Prompt-Response) pairs.
  - **Evaluation & Deployment**: Organizing instruction data loaders, extracting model responses, and evaluating performance metrics for real-world reliability.

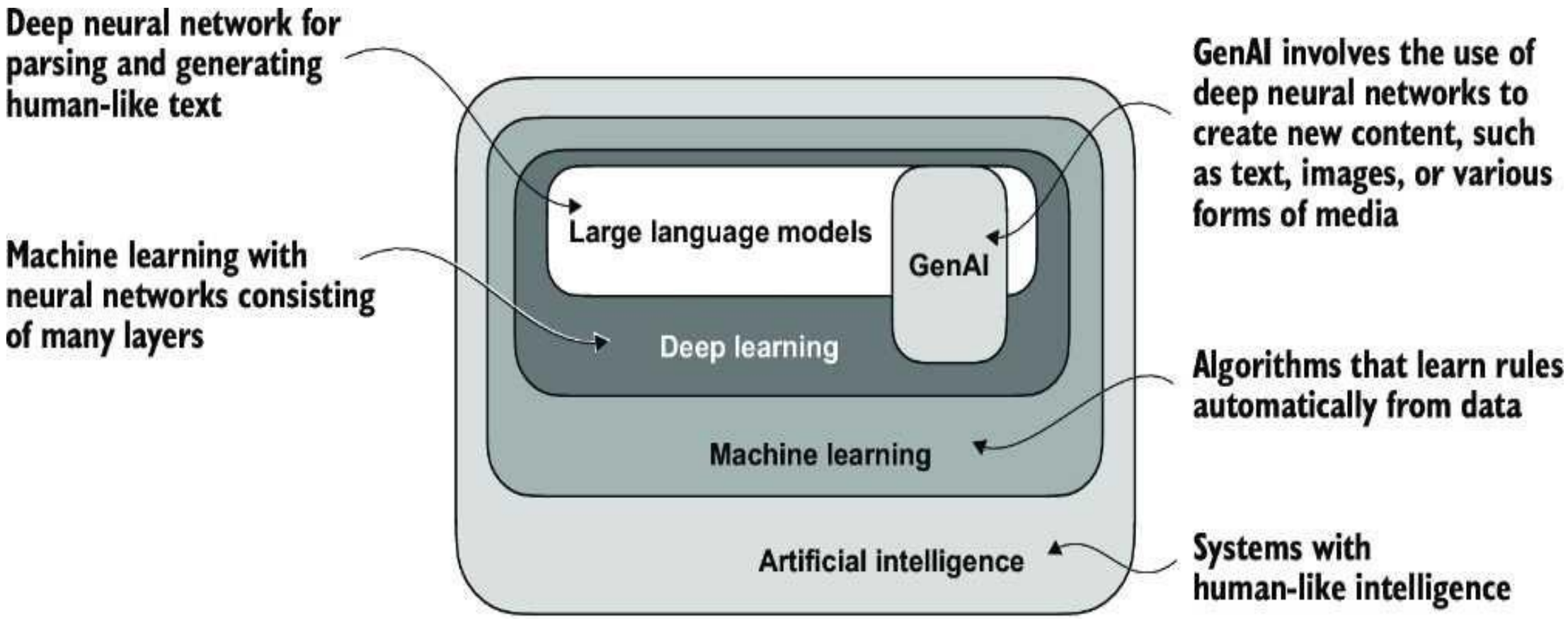
Unit-1: Understanding Large Language Model

- LLMs are trained on vast quantities of text data. This large-scale training allows LLMs to capture deeper contextual information and subtleties of human language compared to previous approaches
- As a result, LLMs have significantly improved performance in a wide range of NLP tasks, including text translation, sentiment analysis, question answering, and many more.
- The success behind LLMs can be attributed to the transformer architecture that underpins many LLMs and the vast amounts of data on which LLMs are trained, allowing them to capture a wide variety of linguistic nuances, contexts, and patterns that would be challenging to encode manually.

What is an LLM?

- An LLM is a neural network designed to understand, generate, and respond to human-like text. These models are deep neural networks trained on massive amounts of text data, sometimes encompassing large portions of the entire publicly available text on the internet.
- The “large” in “large language model” refers to both the model's size in terms of parameters and the immense dataset on which it's trained.
- Models like this often have tens or even hundreds of billions of parameters, which are the adjustable weights in the network that are optimized during training to predict the next word in a sequence.

- Next-word prediction is sensible because it harnesses the inherent sequential nature of language to train models on understanding context, structure, and relationships within text.
- LLMs utilize an architecture called the *transformer*, which allows them to pay selective attention to different parts of the input when making predictions, making them especially adept at handling the nuances and complexities of human language.
- Since LLMs are capable of *generating* text, LLMs are also often referred to as a form of generative artificial intelligence, often abbreviated as *generative AI* or *GenAI.*


Deep neural network for parsing and generating human-like text
Machine learning with neural networks consisting of many layers
GenAI involves the use of deep neural networks to create new content, such as text, images, or various forms of media
Algorithms that learn rules automatically from data
Systems with human-like intelligence
Large language models
GenAI
Deep learning
Machine learning
Artificial intelligence

- The algorithms used to implement AI are the focus of the field of machine learning. Specifically, machine learning involves the development of algorithms that can learn from and make predictions or decisions based on data without being explicitly programmed.
- Ex: spam filter

Applications of LLMs

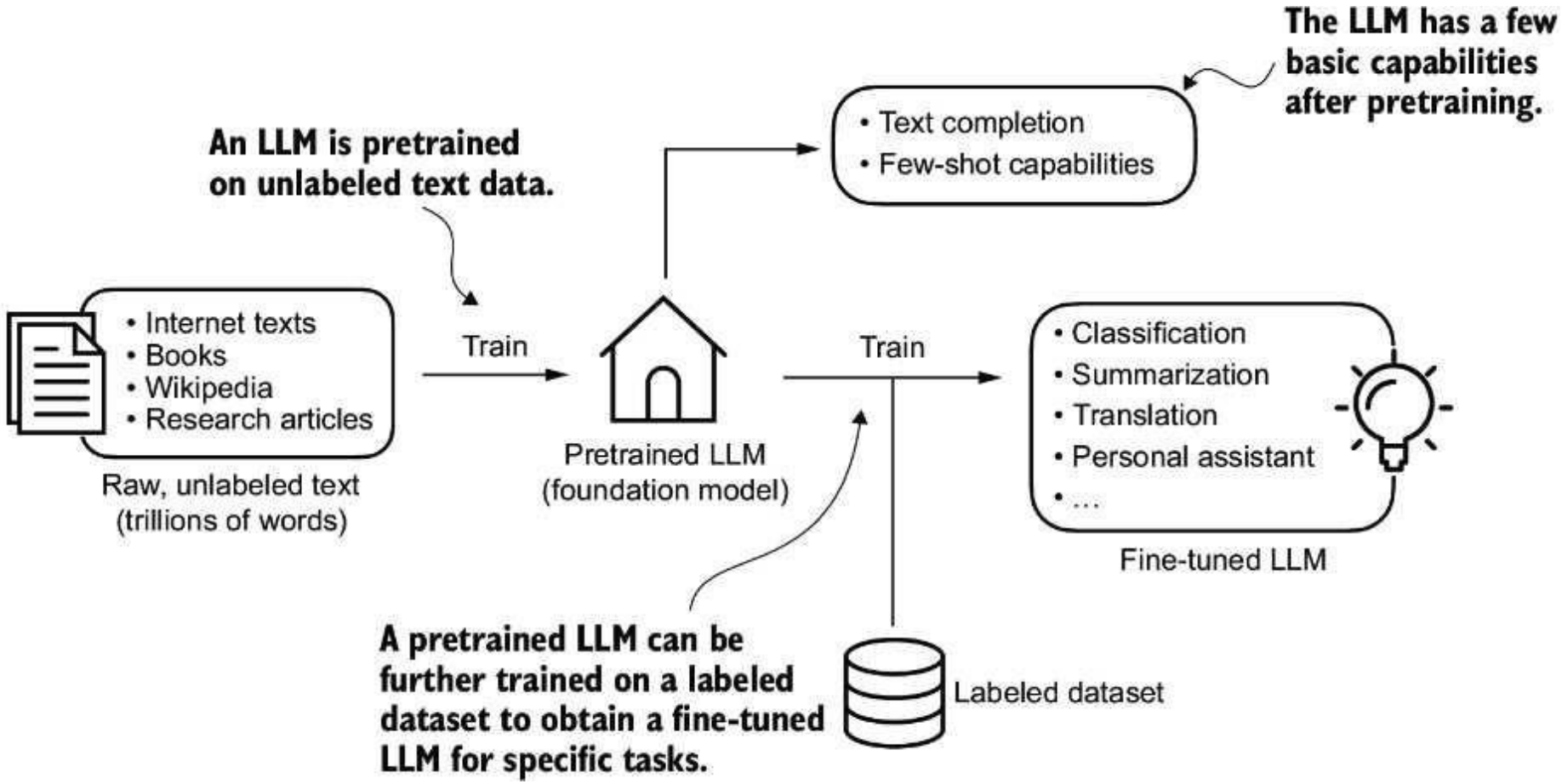
- Today, LLMs are employed for machine translation, generation of novel texts, sentiment analysis, text summarization, and many other tasks. LLMs have recently been used for content creation, such as writing fiction, articles, and even computer code.

Stages of building and using LLMs

- Why should we build our own LLMs? Coding an LLM from the ground up is an excellent exercise to understand its mechanics and limitations.
- Also, it equips us with the required knowledge for pretraining or fine-tuning existing open source LLM architectures to our own domain-specific datasets or tasks.
- Research has shown that when it comes to modeling performance, custom-built LLMs—those tailored for specific tasks or domains—can outperform general-purpose LLMs, such as those provided by ChatGPT, which are designed for a wide array of applications. Ex Logo Designer GPT, Khan Academy GPT, Science Tutor GPT etc

- Using custom-built LLMs offers several advantages, particularly regarding data privacy.
- For instance, companies may prefer not to share sensitive data with third-party LLM providers like OpenAI due to confidentiality concerns.
- Additionally, developing smaller custom LLMs enables deployment directly on customer devices, such as laptops and smartphones, which is something companies like Apple are currently exploring.
- This local implementation can significantly decrease latency and reduce server-related costs. Furthermore, custom LLMs grant developers complete autonomy, allowing them to control updates and modifications to the model as needed

- The general process of creating an LLM includes pretraining and fine-tuning. The “pre” in “pretraining” refers to the initial phase where a model like an LLM is trained on a large, diverse dataset to develop a broad understanding of language.
- This pretrained model then serves as a foundational resource that can be further refined through fine-tuning, a process where the model is specifically trained on a narrower dataset that is more specific to particular tasks or domains. This two-stage training approach consisting of pretraining and fine-tuning


The LLM has a few basic capabilities after pretraining.
• Text completion
• Few-shot capabilities
An LLM is pretrained on unlabeled text data.
• Internet texts
• Books
• Wikipedia
• Research articles
Raw, unlabeled text (trillions of words)
Train
Pretrained LLM (foundation model)
Train
• Classification
• Summarization
• Translation
• Personal assistant
• …
Fine-tuned LLM
A pretrained LLM can be further trained on a labeled dataset to obtain a fine-tuned LLM for specific tasks.
Labeled dataset

- The first step in creating an LLM is to train it on a large corpus of text data, sometimes referred to as *raw* text. Here, “raw” refers to the fact that this data is just regular text without any labeling information. (Filtering may be applied, such as removing formatting characters or documents in unknown languages.)
- This first training stage of an LLM is also known as *pretraining*, creating an initial pretrained LLM, often called a *base* or *foundation model*
- A typical example of such a model is the GPT-3 model (the precursor of the original model offered in ChatGPT). This model is capable of text completion—that is, finishing a half-written sentence provided by a user.

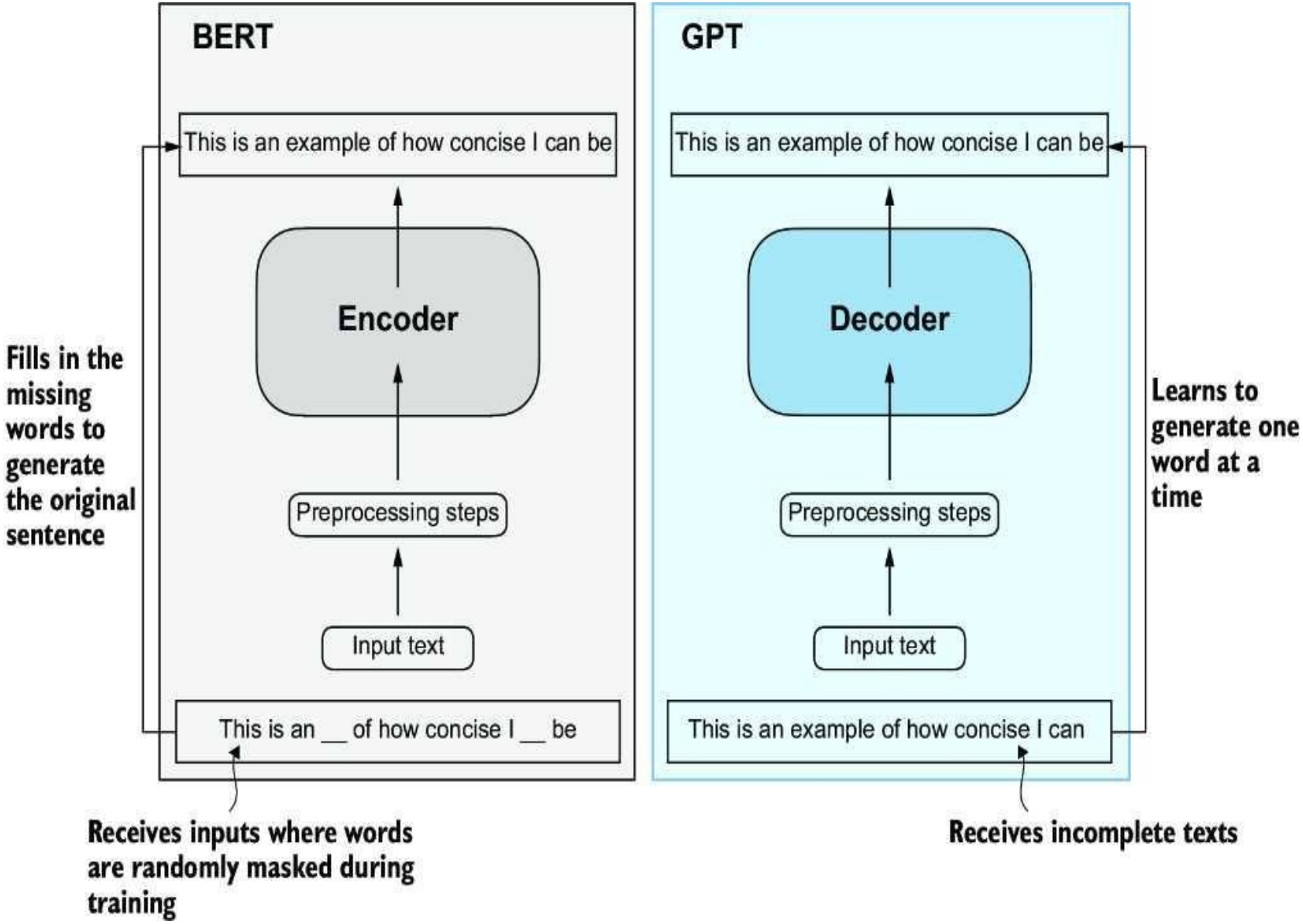
- After obtaining a pretrained LLM from training on large text datasets, where the LLM is trained to predict the next word in the text, we can further train the LLM on labeled data, also known as *fine-tuning.*
- The two most popular categories of fine-tuning LLMs are *instruction fine-tuning* and *classification fine-tuning.*
- In instruction fine-tuning, the labeled dataset consists of instruction and answer pairs, such as a query to translate a text accompanied by the correctly translated text.
- In classification fine-tuning, the labeled dataset consists of texts and associated class labels—for example, emails associated with “spam” and “not spam” labels.

Introducing the transformer architecture

- Most modern LLMs rely on the *transformer* architecture, which is a deep neural network architecture introduced in the 2017 paper "Attention Is All You Need"
- To understand LLMs, we must understand the original transformer, which was developed for machine translation, translating English texts to German and French.
- The transformer architecture consists of two submodules: an encoder and a decoder.
- The encoder module processes the input text and encodes it into a series of numerical representations or vectors that capture the contextual information of the input.

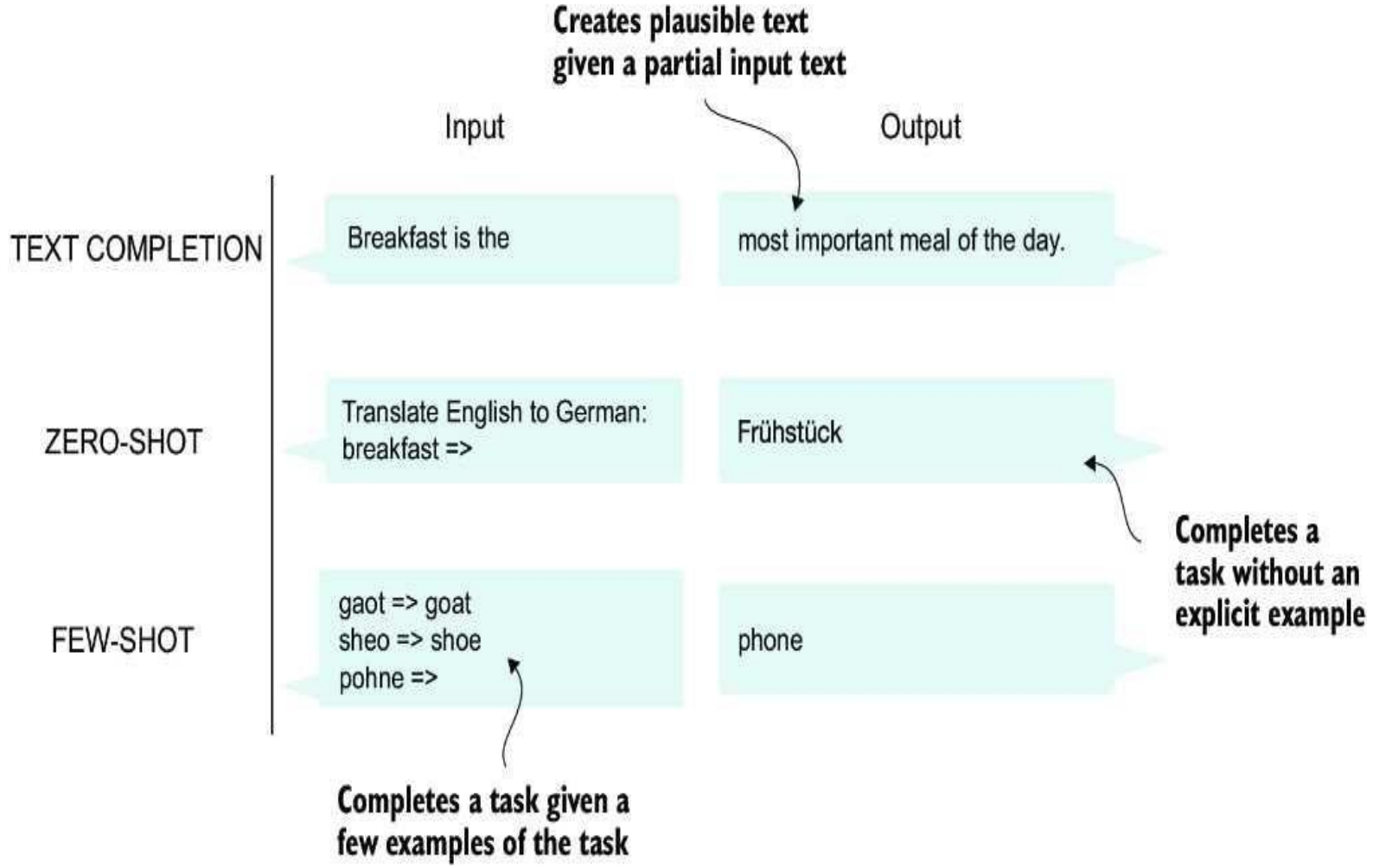
- Then, the decoder module takes these encoded vectors and generates the output text.
- In a translation task, for example, the encoder would encode the text from the source language into vectors, and the decoder would decode these vectors to generate text in the target language.
- Both the encoder and decoder consist of many layers connected by a so-called self-attention mechanism.
- A key component of transformers and LLMs is the self- attention mechanism, which allows the model to weigh the importance of different words or tokens in a sequence relative to each other.
- This mechanism enables the model to capture long-range dependencies and contextual relationships within the input data, enhancing its ability to generate coherent and contextually relevant output

- Later variants of the transformer architecture, such as BERT (short for *bidirectional encoder representations from transformers*) and the various GPT models (short for *generative pretrained transformers*), built on this concept to adapt this architecture for different tasks.
- BERT, which is built upon the original transformer's encoder submodule, differs in its training approach from GPT.
- While GPT is designed for generative tasks, BERT and its variants specialize in masked word prediction, where the model predicts masked or hidden words in a given sentence.
- This unique training strategy equips BERT with strengths in text classification tasks, including sentiment prediction and document categorization


BERT
This is an example of how concise I can be
Encoder
Preprocessing steps
Input text
This is an __ of how concise I __ be
Fills in the missing words to generate the original sentence
Receives inputs where words are randomly masked during training
GPT
This is an example of how concise I can be
Decoder
Preprocessing steps
Input text
This is an example of how concise I can
Learns to generate one word at a time
Receives incomplete texts

- As an application of its capabilities, X (formerly Twitter) uses BERT to detect toxic content.
- GPT, on the other hand, focuses on the decoder portion of the original transformer architecture and is designed for tasks that require generating texts.
- This includes machine translation, text summarization, fiction writing, writing computer code, and more.
- GPT models, primarily designed and trained to perform text completion tasks, also show remarkable versatility in their capabilities

- These models are adapt at executing both **zero- shot** and **few-shot learning** tasks. **Zero-shot** learning refers to the ability to generalize to completely unseen tasks without any prior specific examples. On the other hand, **few- shot** learning involves learning from a minimal number of examples the user provides as input



Utilizing large datasets

- The large training datasets for popular GPT- and BERT-like models represent diverse and comprehensive text corpora encompassing billions of words, which include a vast array of topics and natural and computer languages.

Table 1.1 The pretraining dataset of the popular GPT-3 LLM

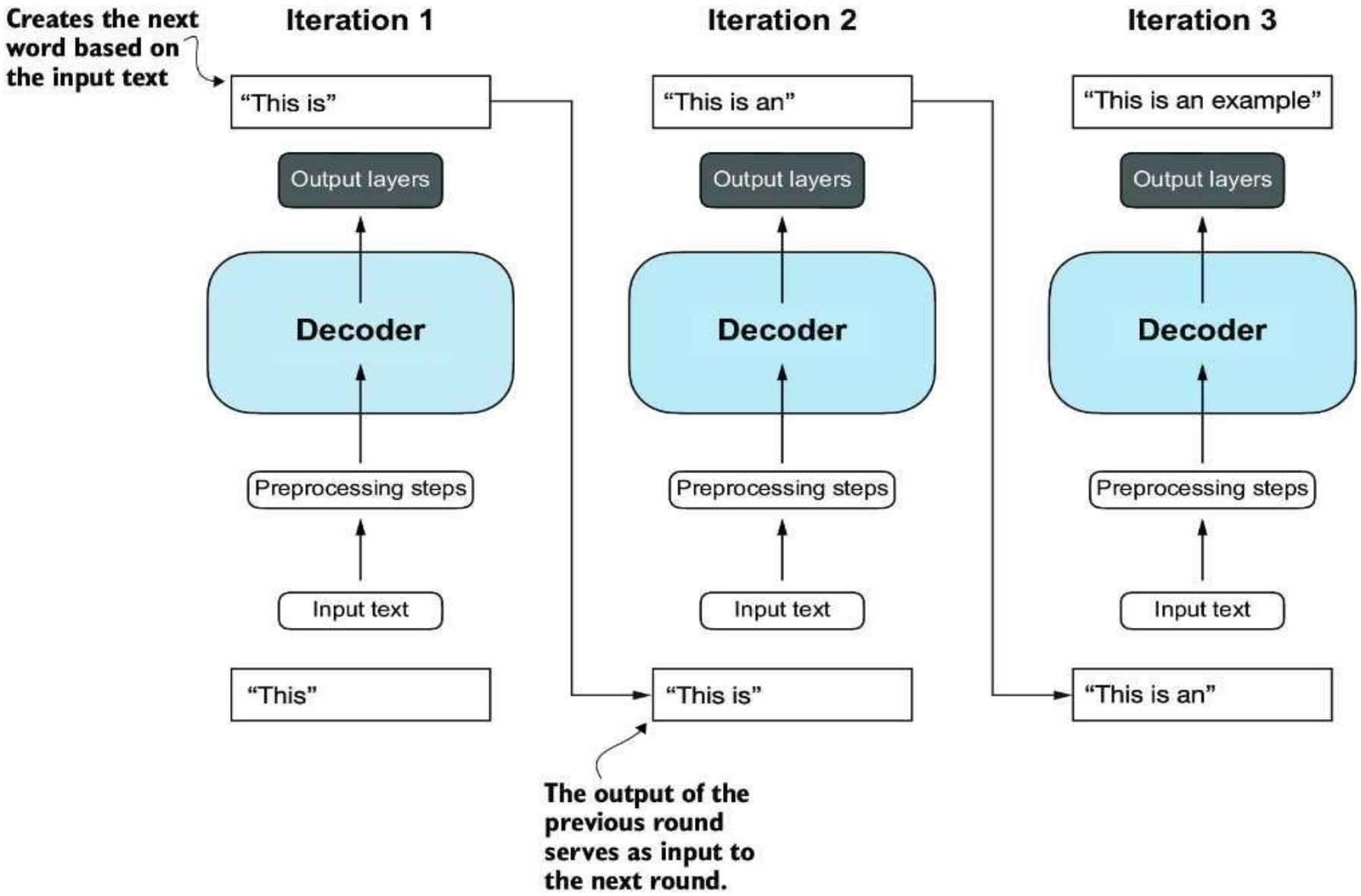
Dataset name	Dataset description	Number of tokens	Proportion in training data
CommonCrawl (filtered)	Web crawl data	410 billion	60%
WebText2	Web crawl data	19 billion	22%
Books1	Internet-based book corpus	12 billion	8%
Books2	Internet-based book corpus	55 billion	8%
Wikipedia	High-quality text	3 billion	3%

- The pretrained nature of these models makes them incredibly versatile for further fine-tuning on downstream tasks, which is why they are also known as base or foundation models.
- Pretraining LLMs requires access to significant resources and is very expensive. For example, the GPT-3 pretraining cost is estimated to be $4.6 million in terms of cloud computing credits
- The good news is that many pretrained LLMs, available as open source models, can be used as general-purpose tools to write, extract, and edit texts that were not part of the training data.
- Also, LLMs can be fine-tuned on specific tasks with relatively smaller datasets, reducing the computational resources needed and improving performance

A closer look at the GPT architecture

- GPT was originally introduced in the paper “Improving Language Understanding by Generative Pre-Training” by Radford et al. from OpenAI.
- GPT-3 is a scaled-up version of this model that has more parameters and was trained on a larger dataset. In addition, the original model offered in ChatGPT was created by fine- tuning GPT-3 on a large instruction dataset using a method from OpenAI's InstructGPT

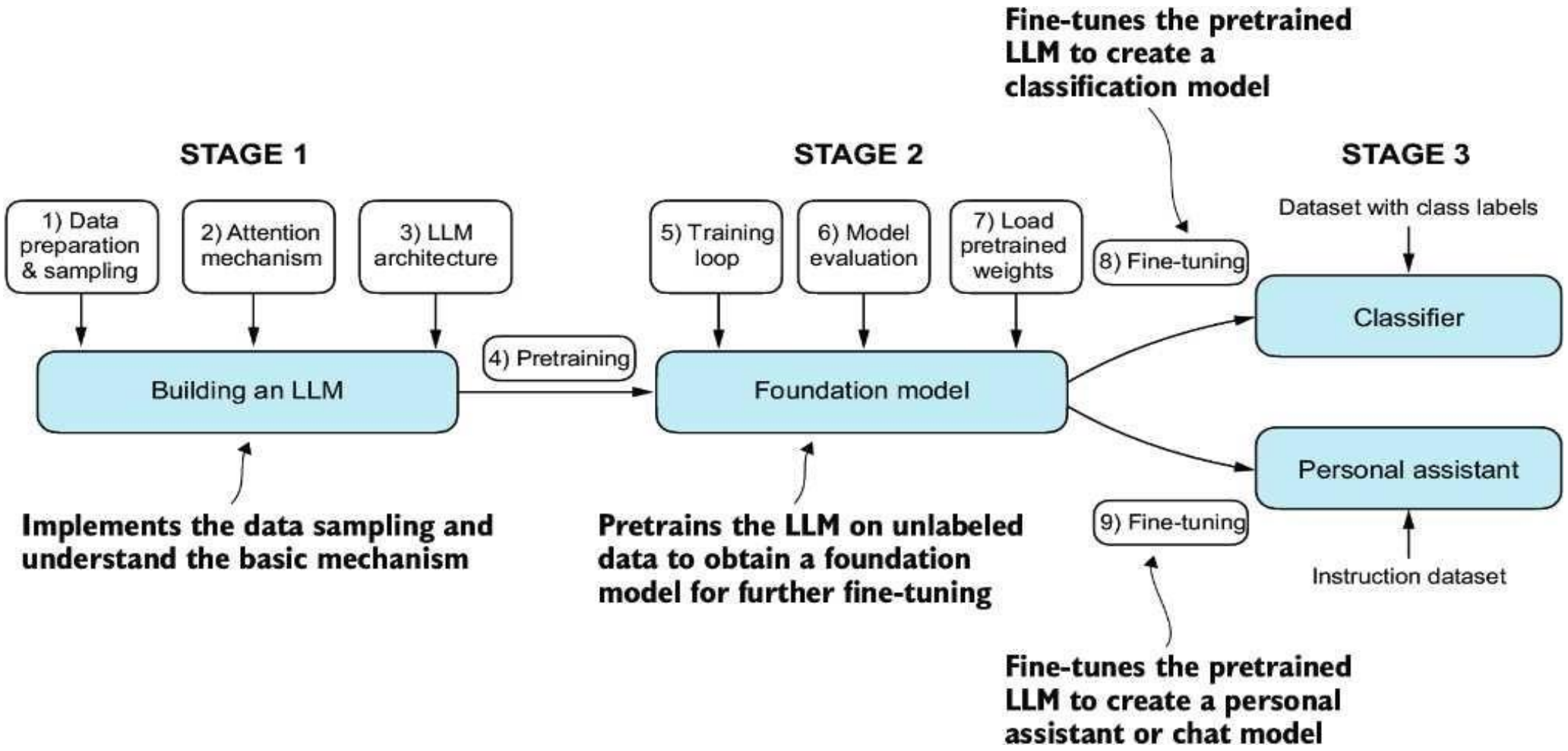
- The next-word prediction task is a form of self-supervised learning, which is a form of self-labeling.
- This means that we don't need to collect labels for the training data explicitly but can use the structure of the data itself
- Since this next-word prediction task allows us to create labels “on the fly,” it is possible to use massive unlabeled text datasets to train LLMs.
- Compared to the original transformer architecture the general GPT architecture is relatively simple. Essentially, it's just the decoder part without the encoder


Creates the next word based on the input text
Iteration 1
Iteration 2
Iteration 3
"This is"
"This is an"
"This is an example"
Output layers
Output layers
Output layers
Decoder
Decoder
Decoder
Preprocessing steps
Preprocessing steps
Preprocessing steps
Input text
Input text
Input text
"This"
"This is"
"This is an"
The output of the previous round serves as input to the next round.

- Since decoder-style models like GPT generate text by predicting text one word at a time, they are considered a type of *autoregressive* model.
- Autoregressive models incorporate their previous outputs as inputs for future predictions. Consequently, in GPT, each new word is chosen based on the sequence that precedes it, which improves the coherence of the resulting text.
- Architectures such as GPT-3 are also significantly larger than the original transformer model. For instance, the original transformer repeated the encoder and decoder blocks six times. GPT-3 has 96 transformer layers and 175 billion parameters in total.

- Although the original transformer model, consisting of encoder and decoder blocks, was explicitly designed for language translation, GPT models—despite their larger yet simpler decoder-only architecture aimed at next-word prediction—are also capable of performing translation tasks.
- The ability to perform tasks that the model wasn't explicitly trained to perform is called an *emergent behavior*
- This capability isn't explicitly taught during training but emerges as a natural consequence of the model's exposure to vast quantities of multilingual data in diverse contexts

Building a large language model



- In stage 1, we will learn about the fundamental data preprocessing steps and code the attention mechanism at the heart of every LLM.
- Next, in stage 2, we will learn how to code and pretrain a GPT-like LLM capable of generating new texts
- We will also go over the fundamentals of evaluating LLMs, which is essential for developing capable NLP systems.
- Pretraining an LLM from scratch is a significant endeavor, demanding thousands to millions of dollars in computing costs for GPT-like models. Therefore, the focus of stage 2 is on implementing training for educational purposes using a small dataset.
- Finally, in stage 3, we will take a pretrained LLM and fine- tune it to follow instructions such as answering queries or classifying texts—the most common tasks in many real- world applications and research.

Summary

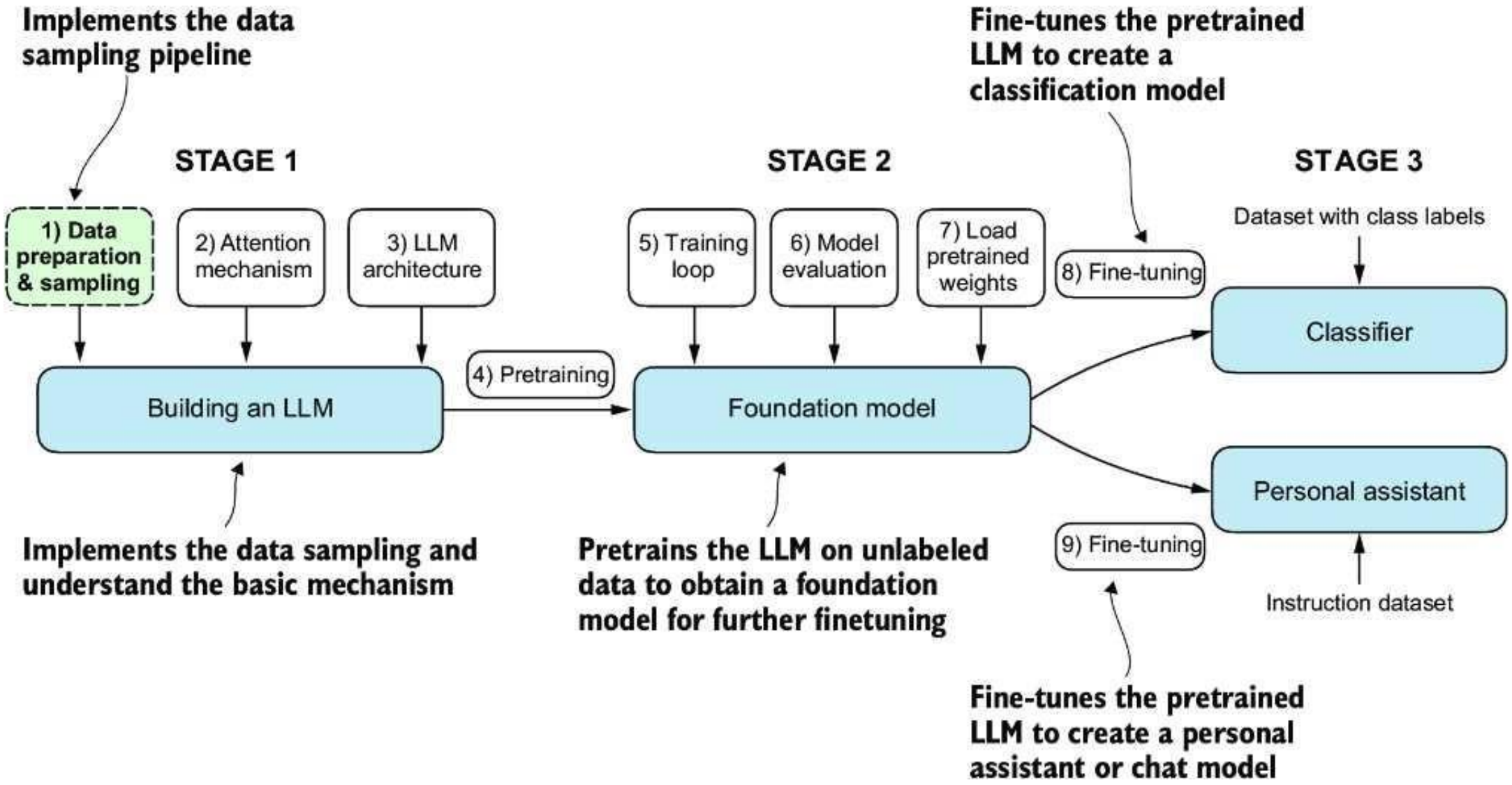
- LLMs have transformed the field of natural language processing, which previously mostly relied on explicit rule-based systems and simpler statistical methods. The advent of LLMs introduced new deep learning- driven approaches that led to advancements in understanding, generating, and translating human language.
- Modern LLMs are trained in two main steps:
  - First, they are pretrained on a large corpus of unlabeled text by using the prediction of the next word in a sentence as a label.
  - Then, they are fine-tuned on a smaller, labeled target dataset to follow instructions or perform classification tasks.
- LLMs are based on the transformer architecture. The key idea of the transformer architecture is an attention mechanism that gives the LLM selective access to the whole input sequence when generating the output one word at a time.

- The original transformer architecture consists of an encoder for parsing text and a decoder for generating text.
- LLMs for generating text and following instructions, such as GPT-3 and ChatGPT, only implement decoder modules, simplifying the architecture.
- Large datasets consisting of billions of words are essential for pretraining LLMs.
- While the general pretraining task for GPT-like models is to predict the next word in a sentence, these LLMs exhibit emergent properties, such as capabilities to classify, translate, or summarize texts.
- Once an LLM is pretrained, the resulting foundation model can be fine- tuned more efficiently for various downstream tasks.
- LLMs fine-tuned on custom datasets can outperform general LLMs on specific tasks.

Thank You

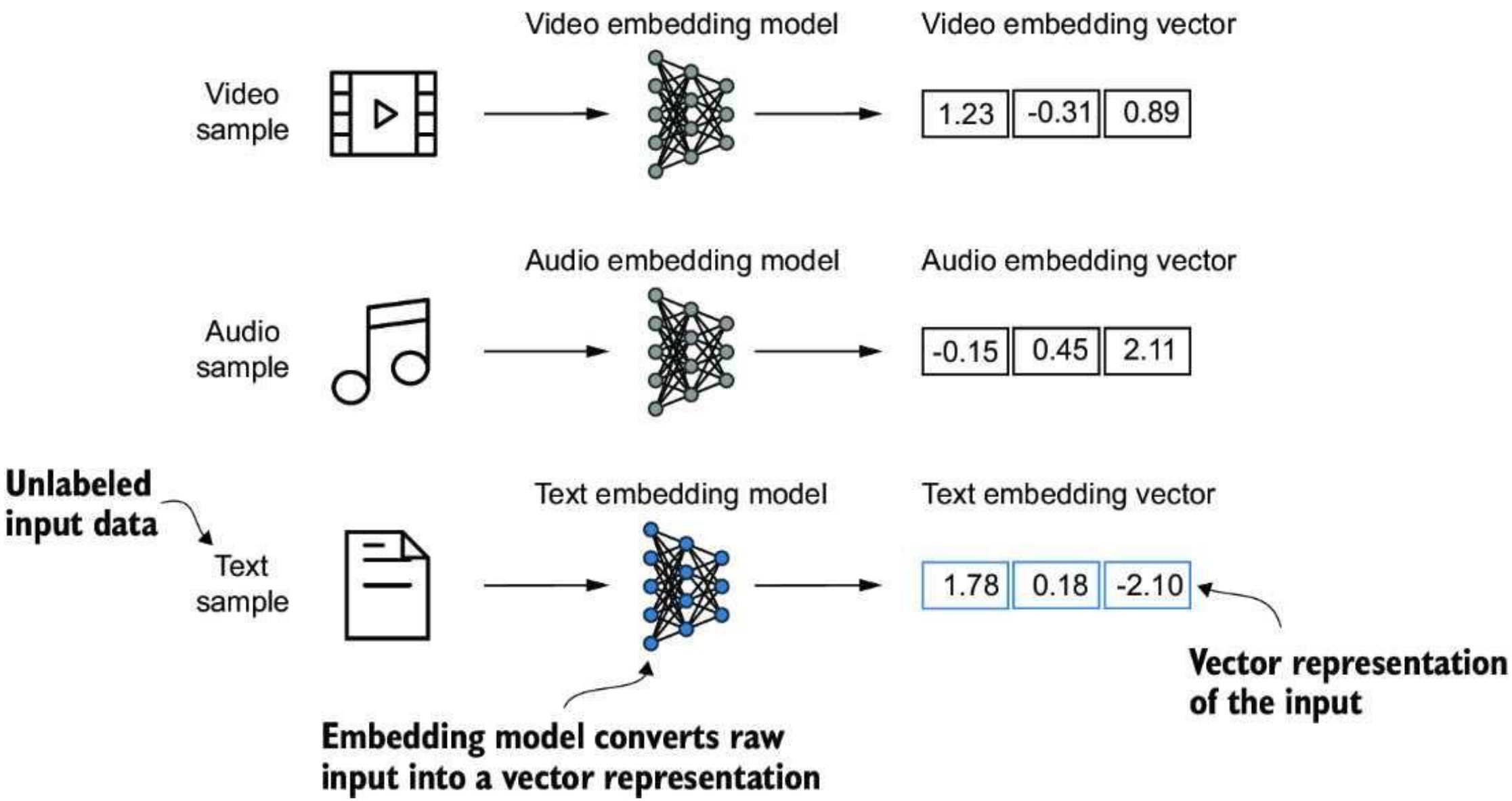
Chapter-2: Working with text data

- we've covered the general structure of large language models (LLMs) and learned that they are pretrained on vast amounts of text
- Specifically, our focus was on decoder-only LLMs based on the transformer architecture, which underlies the models used in ChatGPT and other popular GPT-like LLMs.
- During the pretraining stage, LLMs process text one word at a time.
- Training LLMs with millions to billions of parameters using a next-word prediction task yields models with impressive capabilities.
- These models can then be further finetuned to follow general instructions or perform specific target tasks. But before we can implement and train LLMs, we need to prepare the training dataset


Implements the data sampling pipeline
Fine-tunes the pretrained LLM to create a classification model
STAGE 1
STAGE 2
STAGE 3
1) Data preparation & sampling
2) Attention mechanism
3) LLM architecture
5) Training loop
6) Model evaluation
7) Load pretrained weights
8) Fine-tuning
Dataset with class labels
Classifier
4) Pretraining
Building an LLM
Foundation model
Personal assistant
9) Fine-tuning
Instruction dataset
Implements the data sampling and understand the basic mechanism
Pretrains the LLM on unlabeled data to obtain a foundation model for further finetuning
Fine-tunes the pretrained LLM to create a personal assistant or chat model

Understanding word embeddings

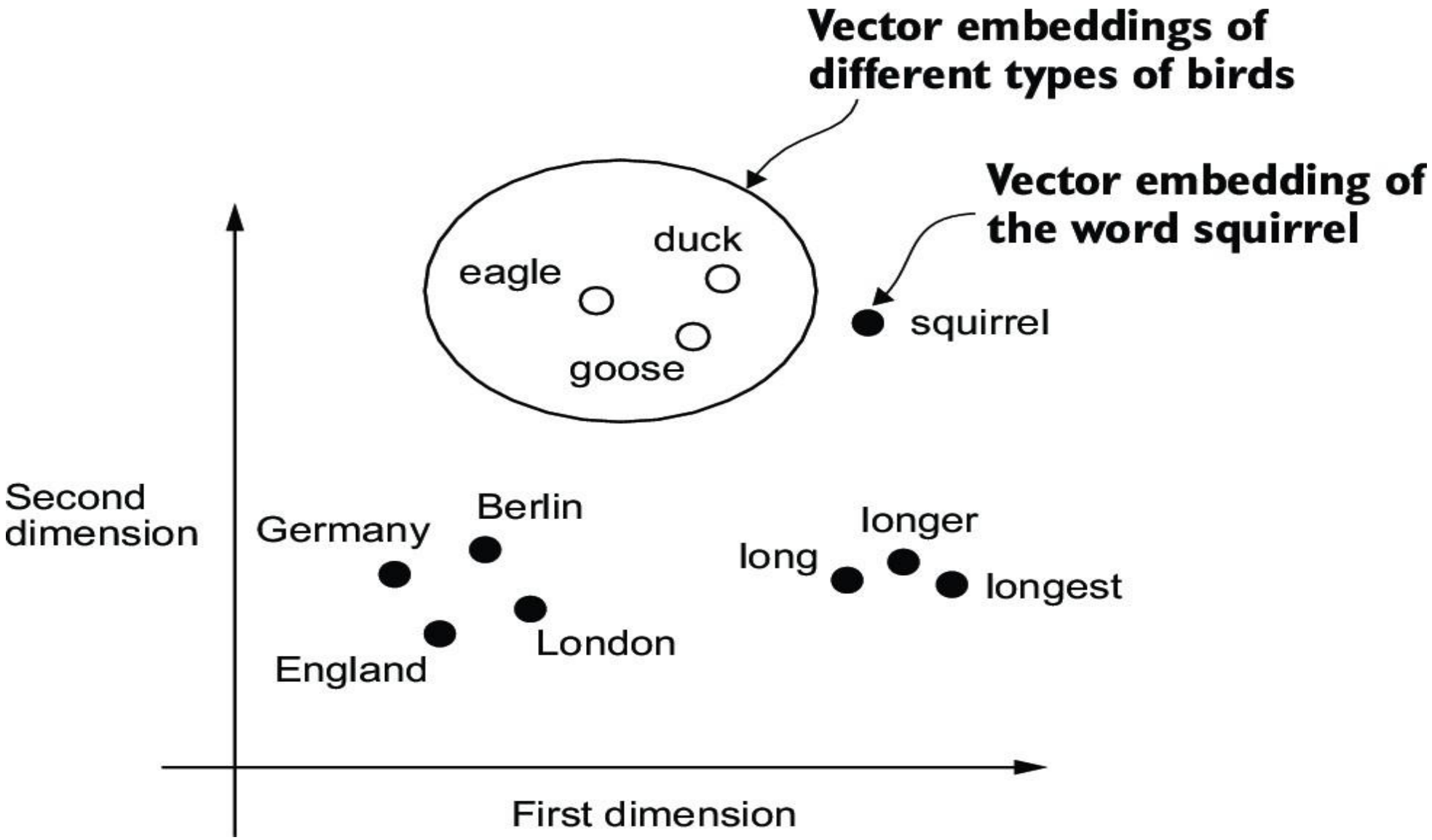
- Deep neural network models, including LLMs, cannot process raw text directly. Since text is categorical, it isn't compatible with the mathematical operations used to implement and train neural networks. Therefore, we need a way to represent words as continuous-valued vectors.
- The concept of converting data into a vector format is often referred to as *embedding*
- Using a specific neural network layer or another pretrained neural network model, we can embed different data types—for example, video, audio, and text, as illustrated in figure


Video embedding model
Video embedding vector
Video sample
1.23
-0.31
0.89
Audio embedding model
Audio embedding vector
Audio sample
-0.15
0.45
2.11
Unlabeled input data
Text embedding model
Text embedding vector
Text sample
1.78
0.18
-2.10
Vector representation of the input
Embedding model converts raw input into a vector representation

- However, it's important to note that different data formats require distinct embedding models.
- For example, an embedding model designed for text would not be suitable for embedding audio or video data.
- At its core, an embedding is a mapping from discrete objects, such as words, images, or even entire documents, to points in a continuous vector space—the primary purpose of embeddings is to convert nonnumeric data into a format that neural networks can process.
- While word embeddings are the most common form of text embedding, there are also embeddings for sentences, paragraphs, or whole documents

- Sentence or paragraph embeddings are popular choices for *retrieval-augmented generation.*
- Retrieval-augmented generation combines generation (like producing text) with retrieval (like searching an external knowledge base) to pull relevant information when generating text, which is a technique that is beyond the scope of this syllabus.
- Since our goal is to train GPT-like LLMs, which learn to generate text one word at a time, we will focus on word embeddings.
- Several algorithms and frameworks have been developed to generate word embeddings. One of the earlier and most popular examples is the *Word2Vec* approach.

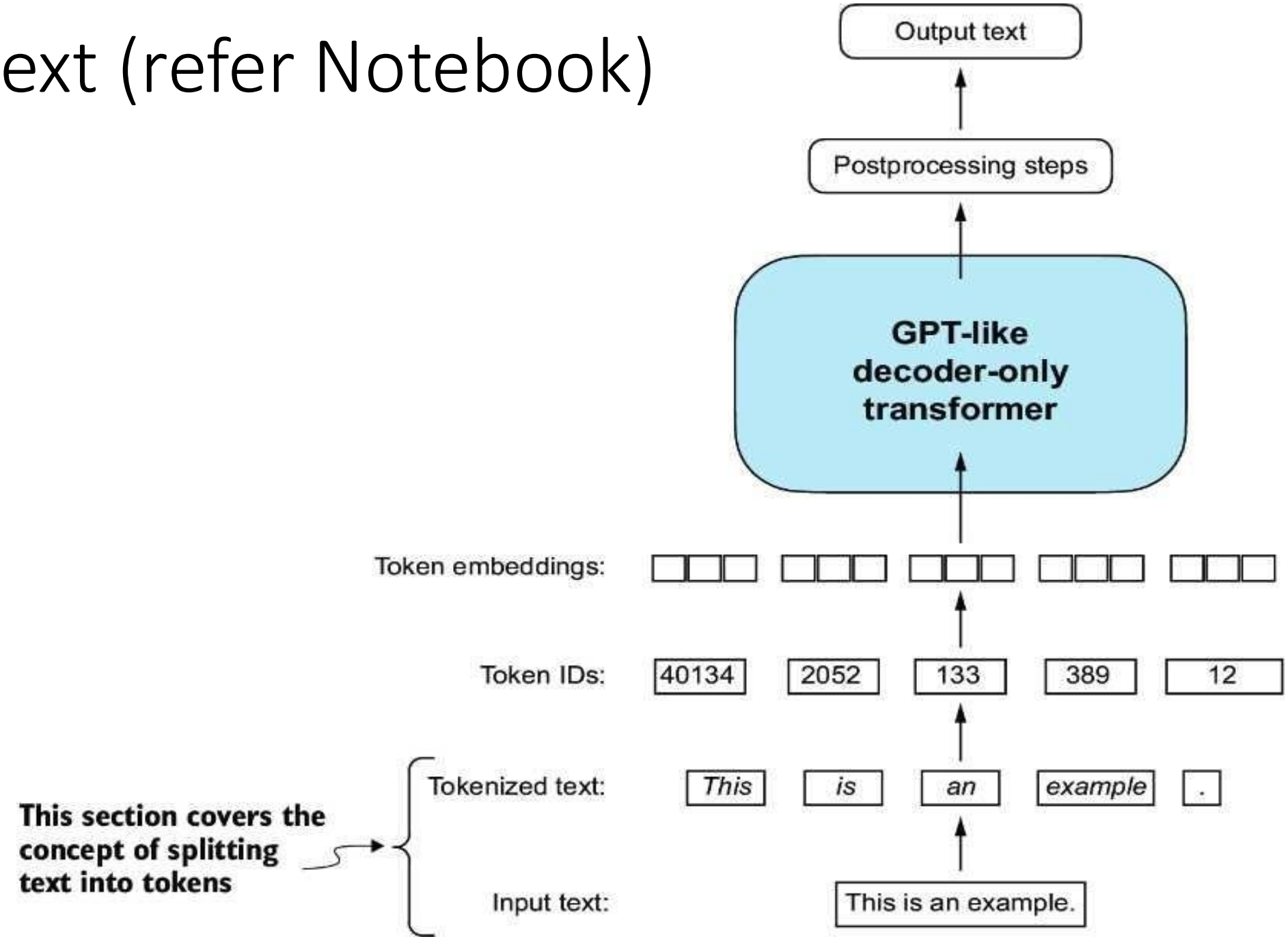
- Word2Vec trained neural network architecture to generate word embeddings by predicting the context of a word given the target word or vice versa. The main idea behind Word2Vec is that words that appear in similar contexts tend to have similar meanings.
- Consequently, when projected into two- dimensional word embeddings for visualization purposes, similar terms are clustered together

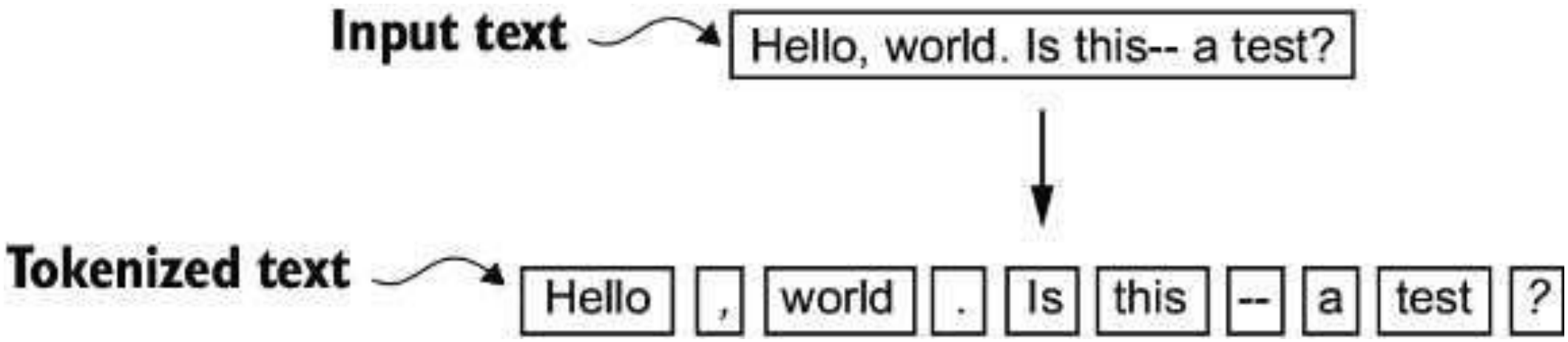

Vector embeddings of different types of birds
Vector embedding of the word squirrel
duck
eagle
goose
squirrel
Second dimension
Germany
Berlin
London
England
long
longer
longest
First dimension

- Word embeddings can have varying dimensions, from one to thousands
- A higher dimensionality might capture more nuanced relationships but at the cost of computational efficiency.
- While we can use pretrained models such as Word2Vec to generate embeddings for machine learning models, LLMs commonly produce their own embeddings that are part of the input layer and are updated during training.
- The advantage of optimizing the embeddings as part of the LLM training instead of using Word2Vec is that the embeddings are optimized to the specific task and data at hand

- Unfortunately, high-dimensional embeddings present a challenge for visualization because our sensory perception and common graphical representations are inherently limited to three dimensions or fewer
- However, when working with LLMs, we typically use embeddings with a much higher dimensionality. For both GPT-2 and GPT-3, the embedding size (often referred to as the dimensionality of the model's hidden states) varies based on the specific model variant and size.
- It is a tradeoff between performance and efficiency. The smallest GPT-2 models (117M and 125M parameters) use an embedding size of 768 dimensions to provide concrete examples. The largest GPT-3 model (175B parameters) uses an embedding size of 12,288 dimensions.

Tokenizing text (refer Notebook)



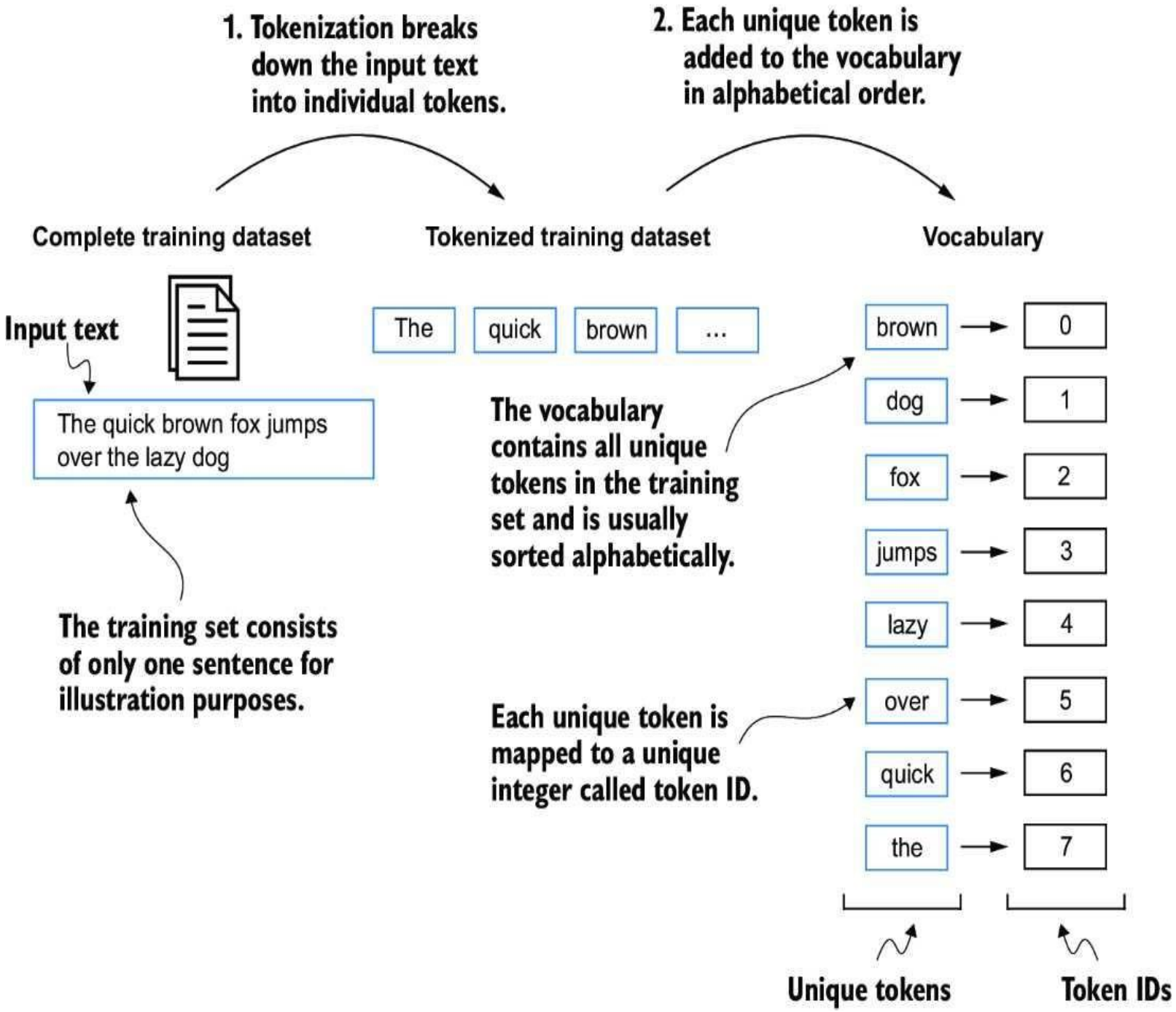

Input text
Hello, world. Is this-- a test?
Tokenized text
Hello
,
world
.
Is
this
--
a
test
?

- Now that we have a basic tokenizer working, let's apply it to Edith Wharton's entire short story (refer Jupyter Notebook for conversion)

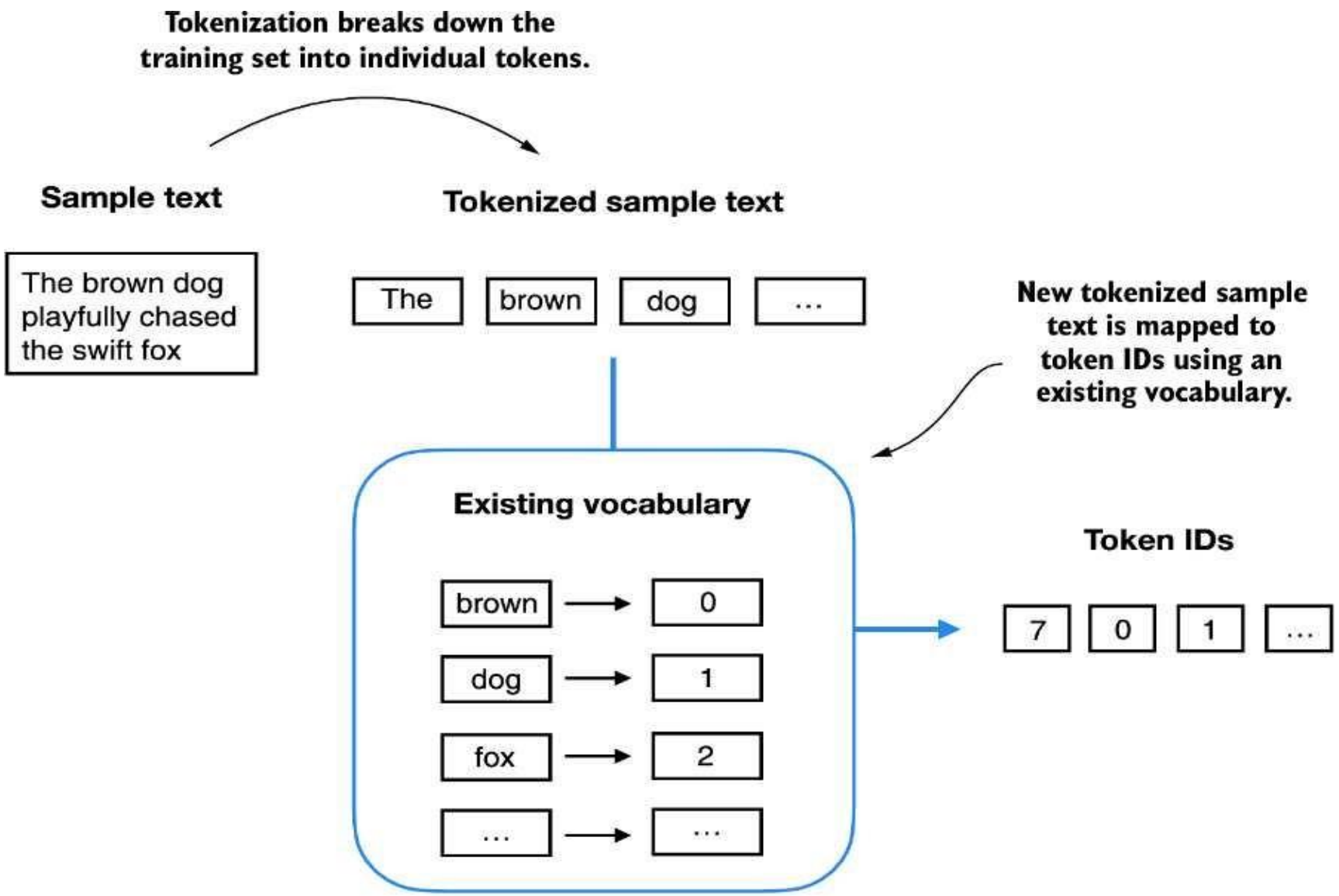
Converting tokens into token IDs

- let's convert these tokens from a Python string to an integer representation to produce the token IDs. This conversion is an intermediate step before converting the token IDs into embedding vectors.
- To map the previously generated tokens into token IDs, we have to build a vocabulary first.
- This vocabulary defines how we map each unique word and special character to a unique integer

- We build a vocabulary by tokenizing the entire text in a training dataset into individual tokens.
- These individual tokens are then sorted alphabetically, and duplicate tokens are removed.
- The unique tokens are then aggregated into a vocabulary that defines a mapping from each unique token to a unique integer value.
- The depicted vocabulary is purposefully small and contains no punctuation or special characters for simplicity.

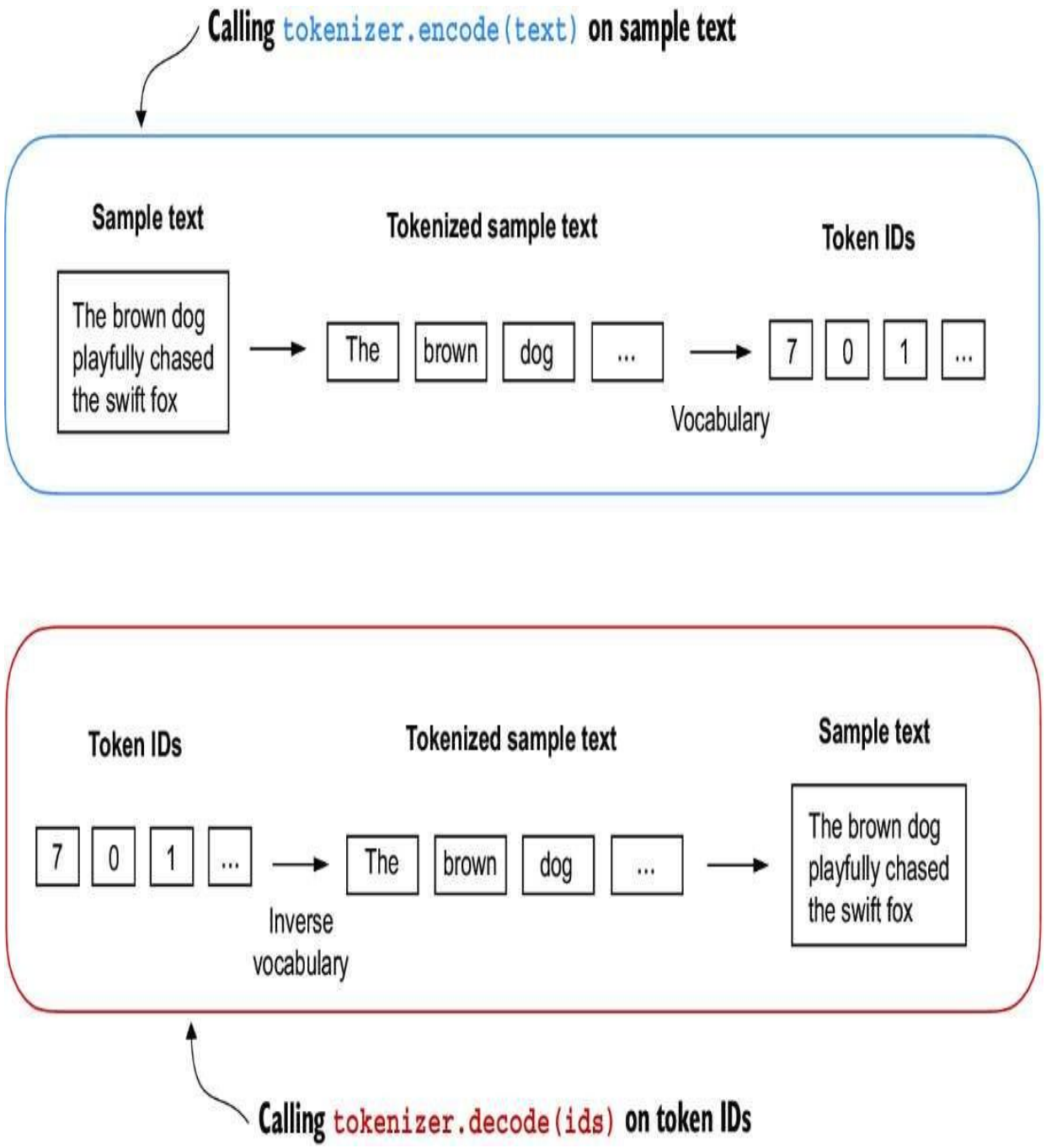


Coming from notebook



- When we want to convert the outputs of an LLM from numbers back into text, we need a way to turn token IDs into text.
- For this, we can create an inverse version of the vocabulary that maps token IDs back to the corresponding text tokens.

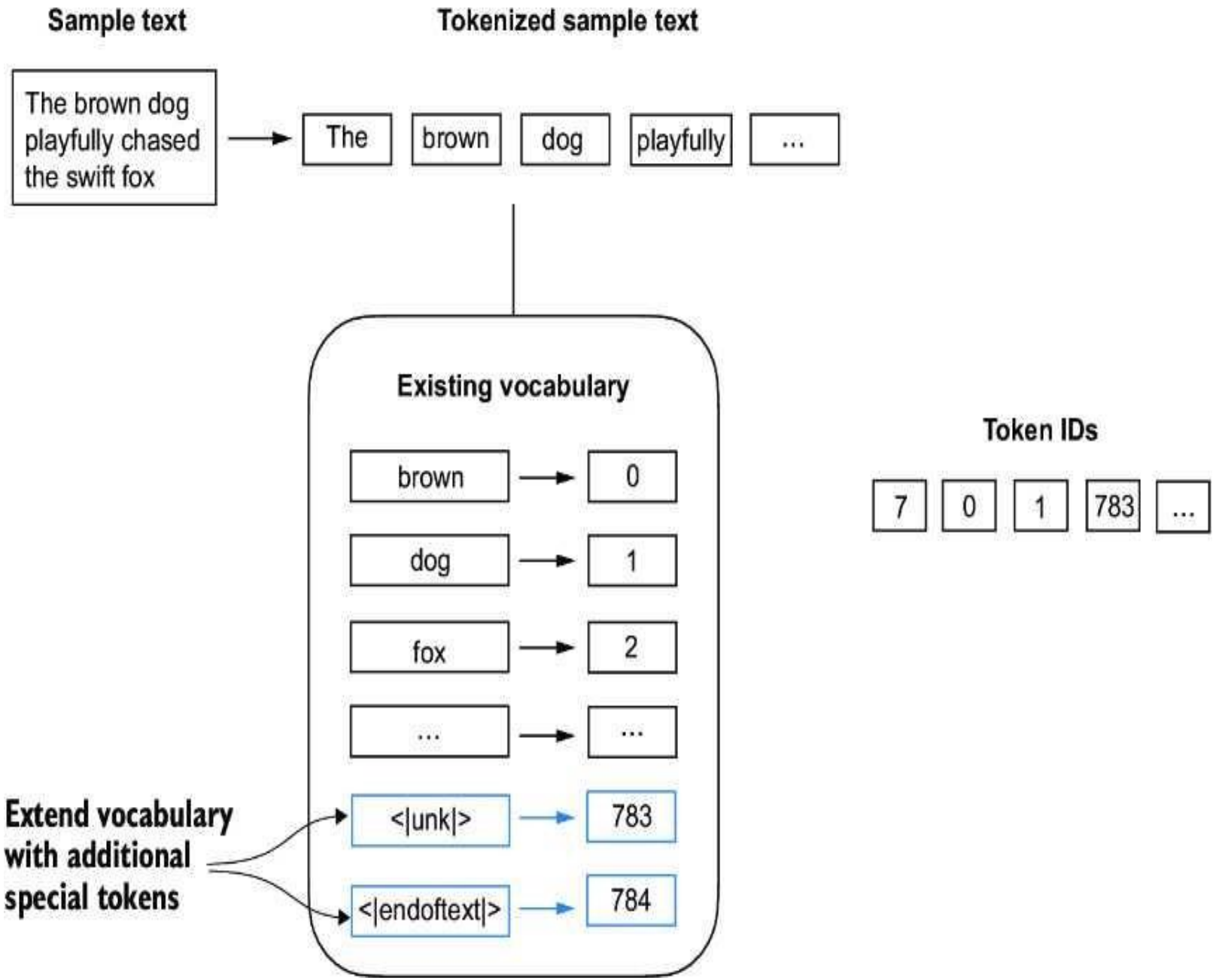
- Tokenizer implementations share two common methods: an encode method and a decode method.
- The encode method takes in the sample text, splits it into individual tokens, and converts the tokens into token IDs via the vocabulary.
- The decode method takes in token IDs, converts them back into text tokens, and concatenates the text tokens into natural text.



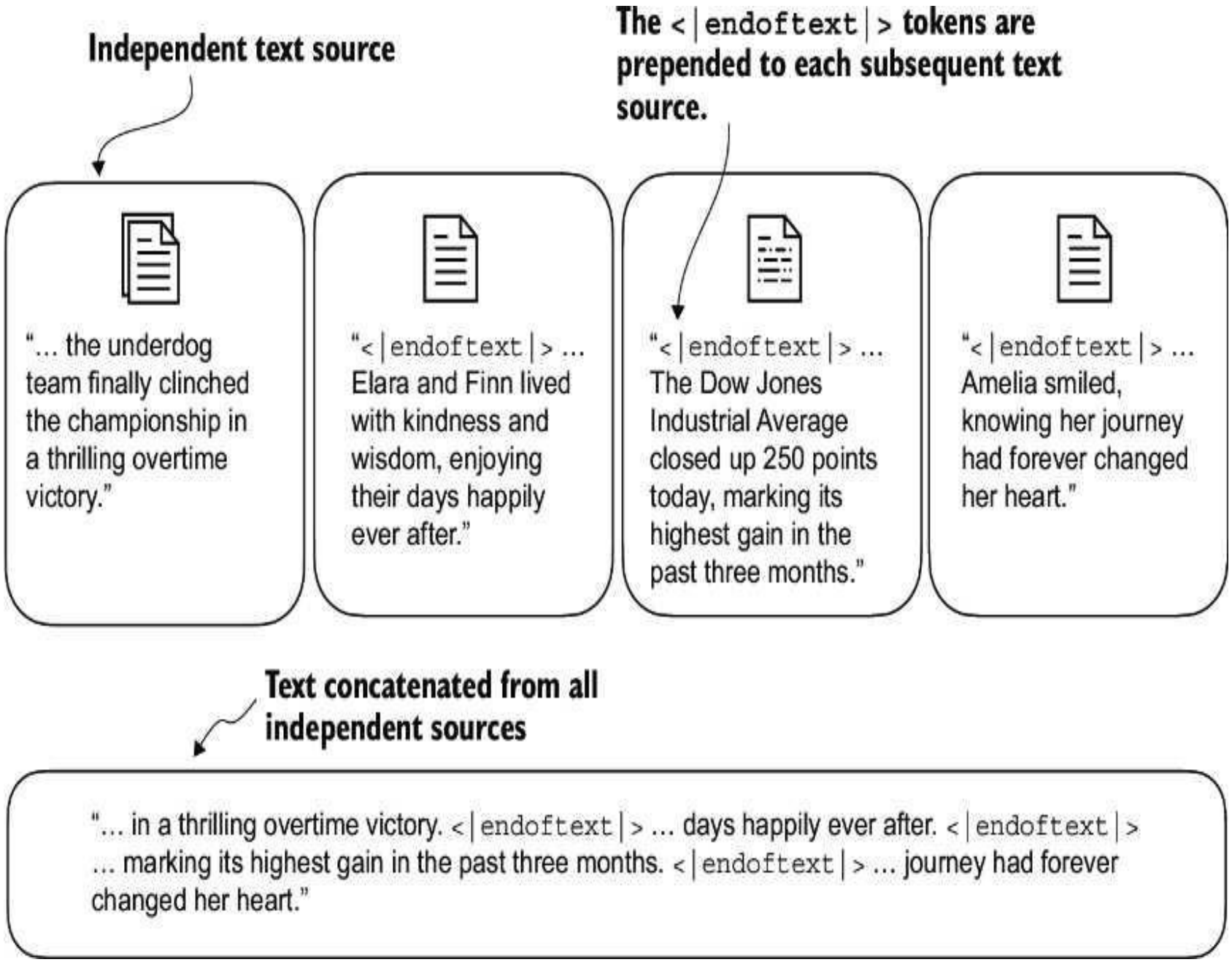
Adding special context tokens

- We need to modify the tokenizer to handle unknown words. We also need to address the usage and addition of special context tokens that can enhance a model's understanding of context or other relevant information in the text.
- These special tokens can include markers for unknown words and document boundaries, for example.
- In particular, we will modify the vocabulary and tokenizer, SimpleTokenizerV2, to support two new tokens, <|unk|> and <|endoftext|>, as

- We add special tokens to a vocabulary to deal with certain contexts.
- For instance, we add an <|unk|> token to represent new and unknown words that were not part of the training data and thus not part of the existing vocabulary.
- Furthermore, we add an <|endoftext|> token that we can use to separate two unrelated text sources



- when training GPT-like LLMs on multiple independent documents or books, it is common to insert a token before each document or book that follows a previous text source.



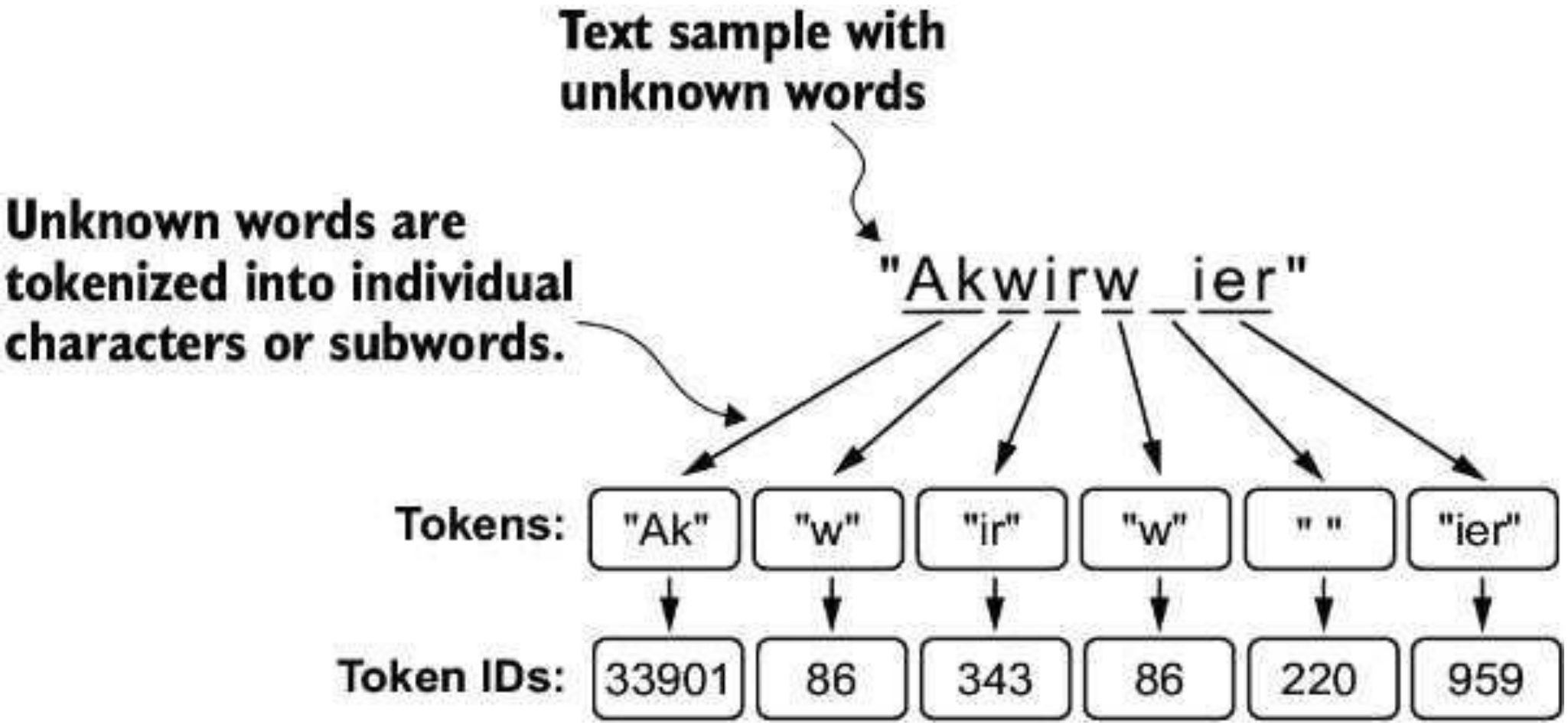
Depending on the LLM, some researchers also consider additional special tokens such as the following:

- [BOS] (beginning of sequence) —This token marks the start of a text. It signifies to the LLM where a piece of content begins.
- [EOS] (end of sequence) —This token is positioned at the end of a text and is especially useful when concatenating multiple unrelated texts, similar to <|endoftext|>. For instance, when combining two different Wikipedia articles or books, the [EOS] token indicates where one ends and the next begins.
- [PAD] (padding) —When training LLMs with batch sizes larger than one, the batch might contain texts of varying lengths. To ensure all texts have the same length, the shorter texts are extended or “padded” using the [PAD] token, up to the length of the longest text in the batch.

Byte pair encoding(BPE)

- The BPE tokenizer was used to train LLMs such as GPT-2, GPT-3, and the original model used in ChatGPT.
- Since implementing BPE can be relatively complicated, we will use an existing Python open source library called *tiktoken* (https://github.com/openai/tiktoken), which implements the BPE algorithm very efficiently
- First, the <|endoftext|> token is assigned a relatively large token ID, namely, 50256.
- In fact, the BPE tokenizer, which was used to train models such as GPT-2, GPT-3, and the original model used in ChatGPT, has a total vocabulary size of 50,257, with <|endoftext|> being assigned the largest token ID.

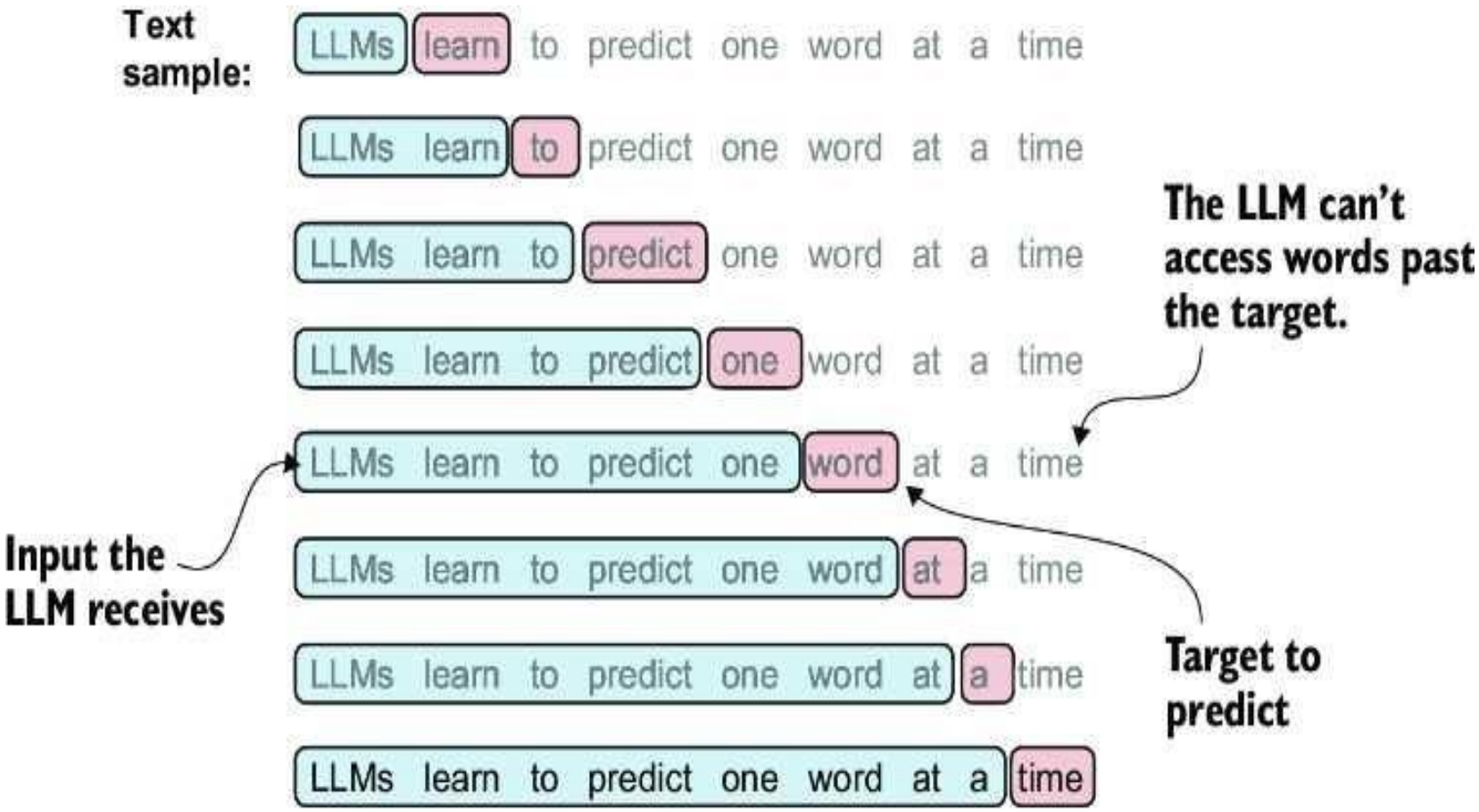
- Second, the BPE tokenizer encodes and decodes unknown words, such as someunknownPlace, correctly. The BPE tokenizer can handle any unknown word. How does it achieve this without using <|unk|> tokens?
- The algorithm underlying BPE breaks down words that aren't in its predefined vocabulary into smaller subword units or even individual characters, enabling it to handle out-of- vocabulary words.
- So, thanks to the BPE algorithm, if the tokenizer encounters an unfamiliar word during tokenization, it can represent it as a sequence of subword tokens or characters.


Text sample with unknown words
Unknown words are tokenized into individual characters or subwords.
"Akwirw ier"
Tokens:
"Ak"
"w"
"ir"
"w"
" "
"ier"
Token IDs:
33901
86
343
86
220
959

- The ability to break down unknown words into individual characters ensures that the tokenizer and, consequently, the LLM that is trained with it can process any text, even if it contains words that were not present in its training data.
- A detailed discussion and implementation of **BPE** is out of the scope of this book, but **in short, it builds its vocabulary by iteratively merging frequent characters into subwords and frequent subwords into words**.
- For example, BPE starts with adding all individual single characters to its vocabulary ("a," "b," etc.).
- In the **next stage, it merges character combinations that frequently occur together into subwords**.
- For example, "**d" and "e" may be merged into the subword "de,"** which is common in many English words like "define," "depend," "made," and "hidden." The merges are determined by a frequency cutoff.

2.6 Data sampling with a sliding window

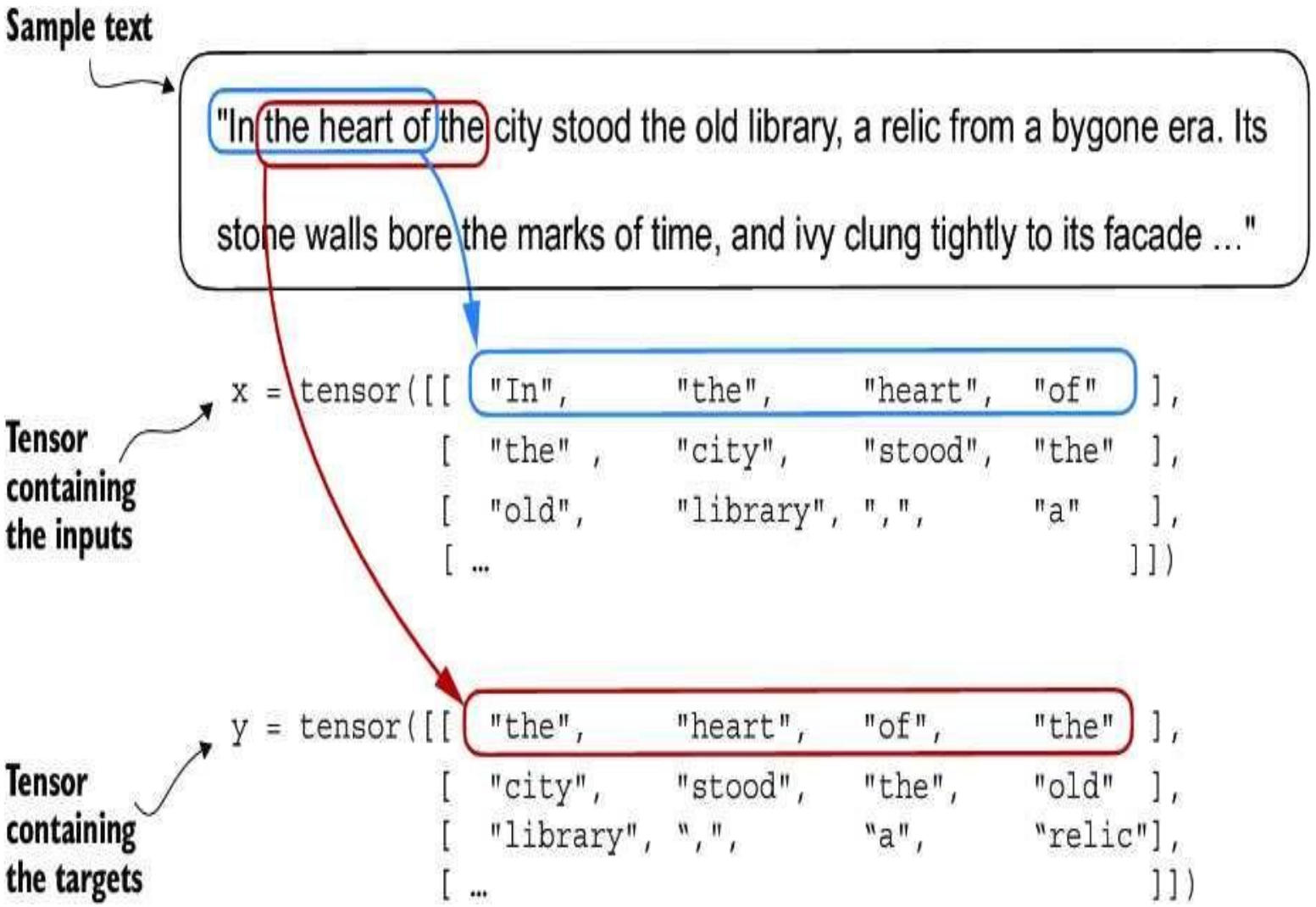
- The next step in creating the embeddings for the LLM is to generate the input–target pairs required for training an LLM.



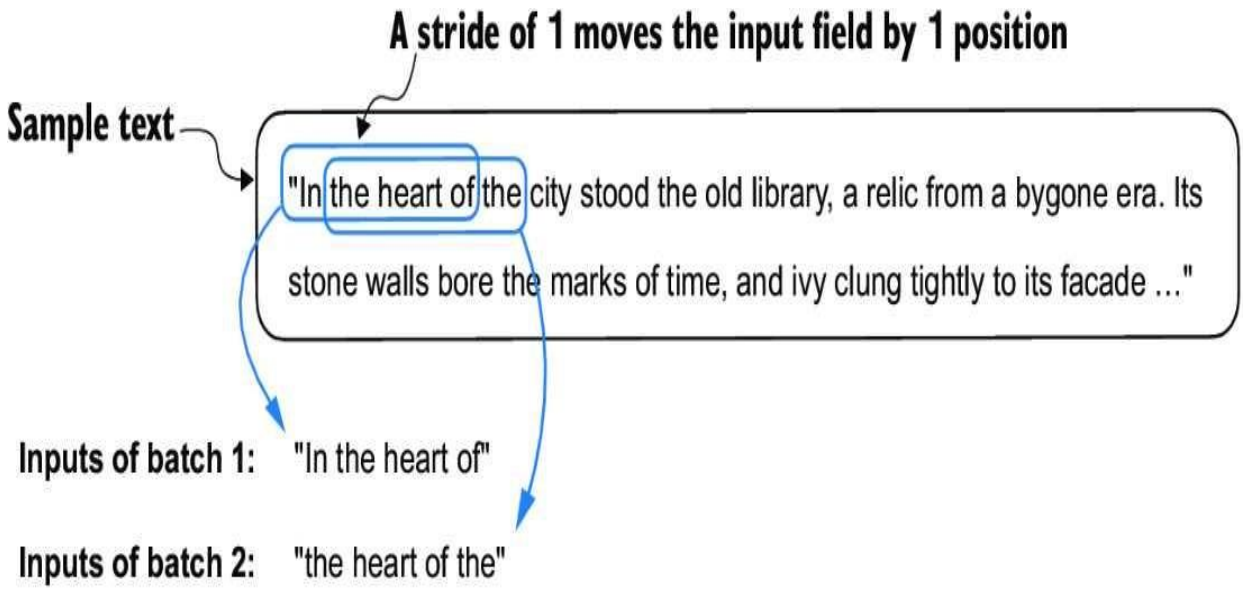
- Given a text sample, extract input blocks as subsamples that serve as input to the LLM, and the **LLM's prediction task during training is to predict the next word that follows the input block**.
- **During training**, **we mask out all words that are past the target**. Note that the text shown in this figure must undergo tokenization before the LLM can process it; however, this figure omits the tokenization step for clarity.
- Let's implement a data loader that fetches the input–target pairs in above figure from the training dataset using a sliding window approach.(➔)

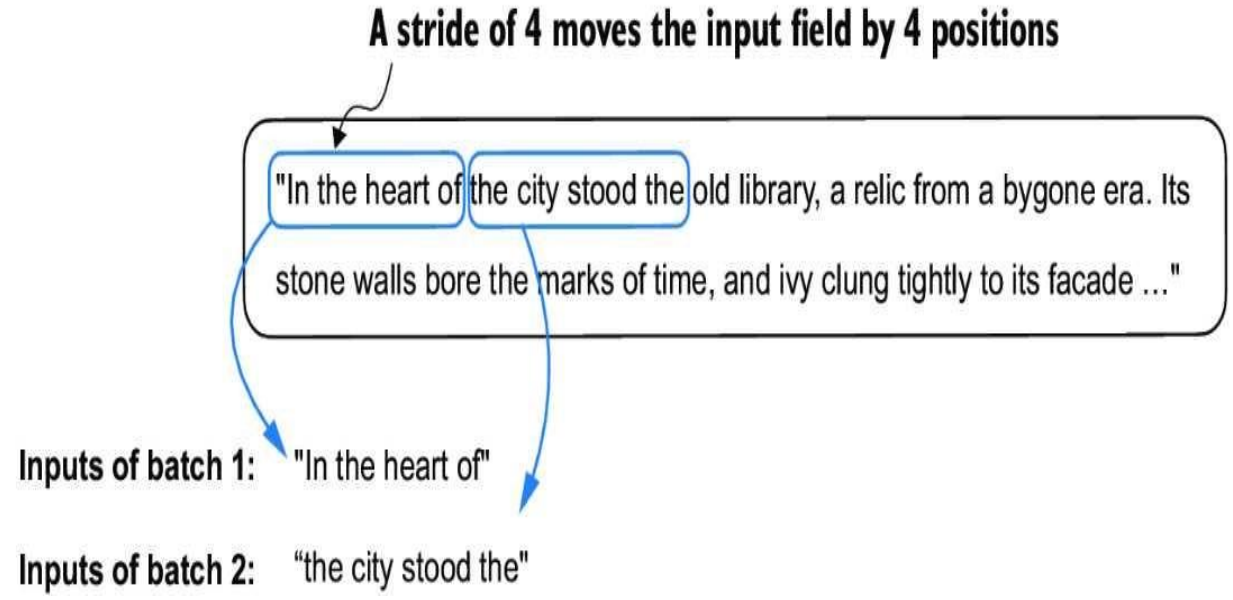
- There's only one more task before we can turn the tokens into embeddings: **implementing an efficient data loader that iterates over the input dataset and returns the inputs and targets as PyTorch tensors, which can be thought of as multidimensional arrays.**
- In particular, we are interested in **returning two tensors**: an **input tensor containing the text that the LLM sees** and a **target tensor that includes the targets for the LLM to predict** as shown in below figure

- To implement efficient data loaders, we collect the inputs in a tensor, x, where each row represents one input context.
- A second tensor, y, contains the corresponding prediction targets (next words), which are created by shifting the input by one position.



- While the figure shows the tokens in string format for illustration purposes, the code implementation will operate on token IDs directly since the encode method of the BPE tokenizer performs both tokenization and conversion into token IDs as a single step.

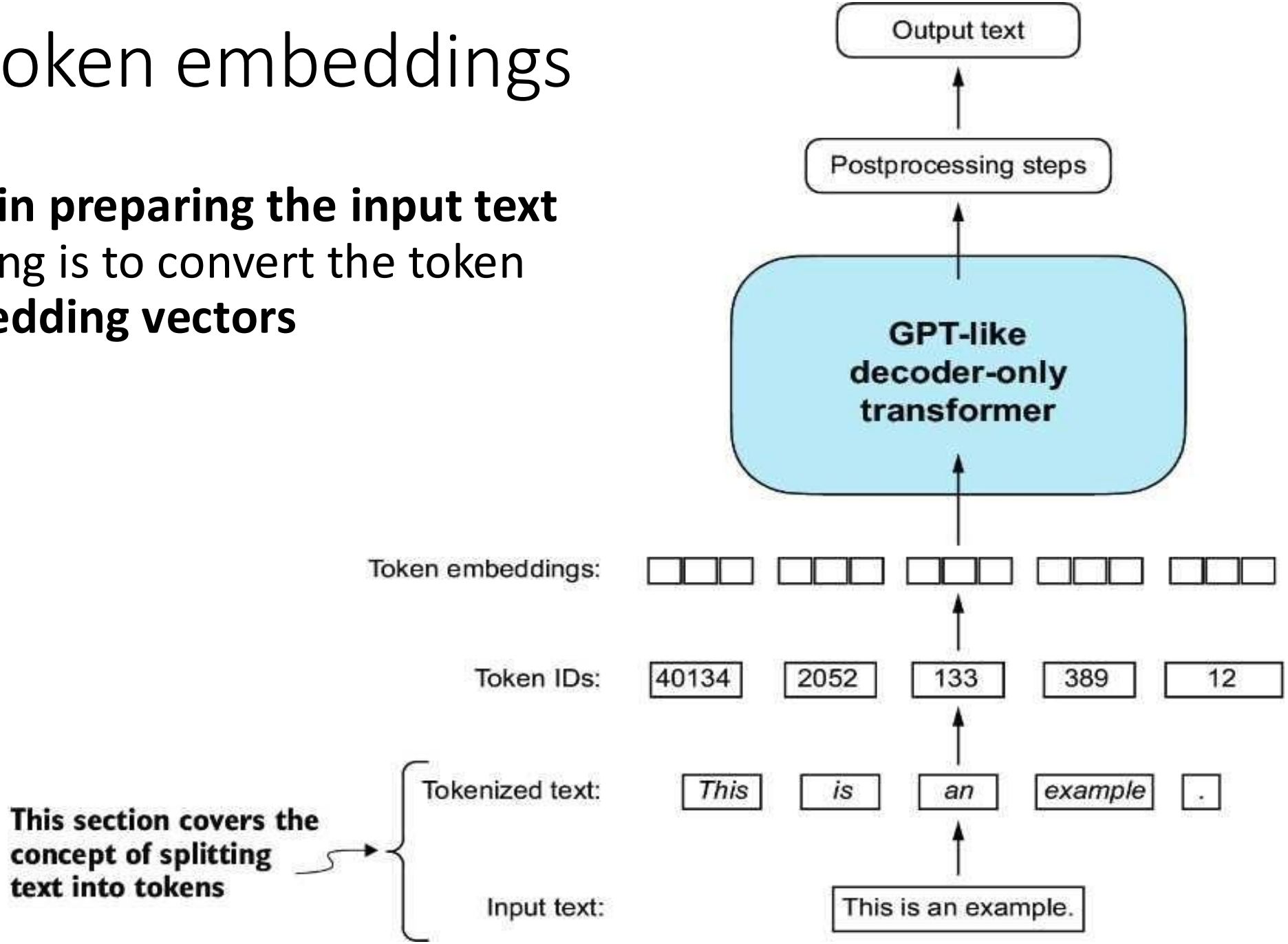

A stride of 1 moves the input field by 1 position
Sample text
"In the heart of the city stood the old library, a relic from a bygone era. Its stone walls bore the marks of time, and ivy clung tightly to its facade ..."
Inputs of batch 1: "In the heart of"
Inputs of batch 2: "the heart of the"


A stride of 4 moves the input field by 4 positions
"In the heart of the city stood the old library, a relic from a bygone era. Its stone walls bore the marks of time, and ivy clung tightly to its facade ..."
Inputs of batch 1: "In the heart of"
Inputs of batch 2: "the city stood the"

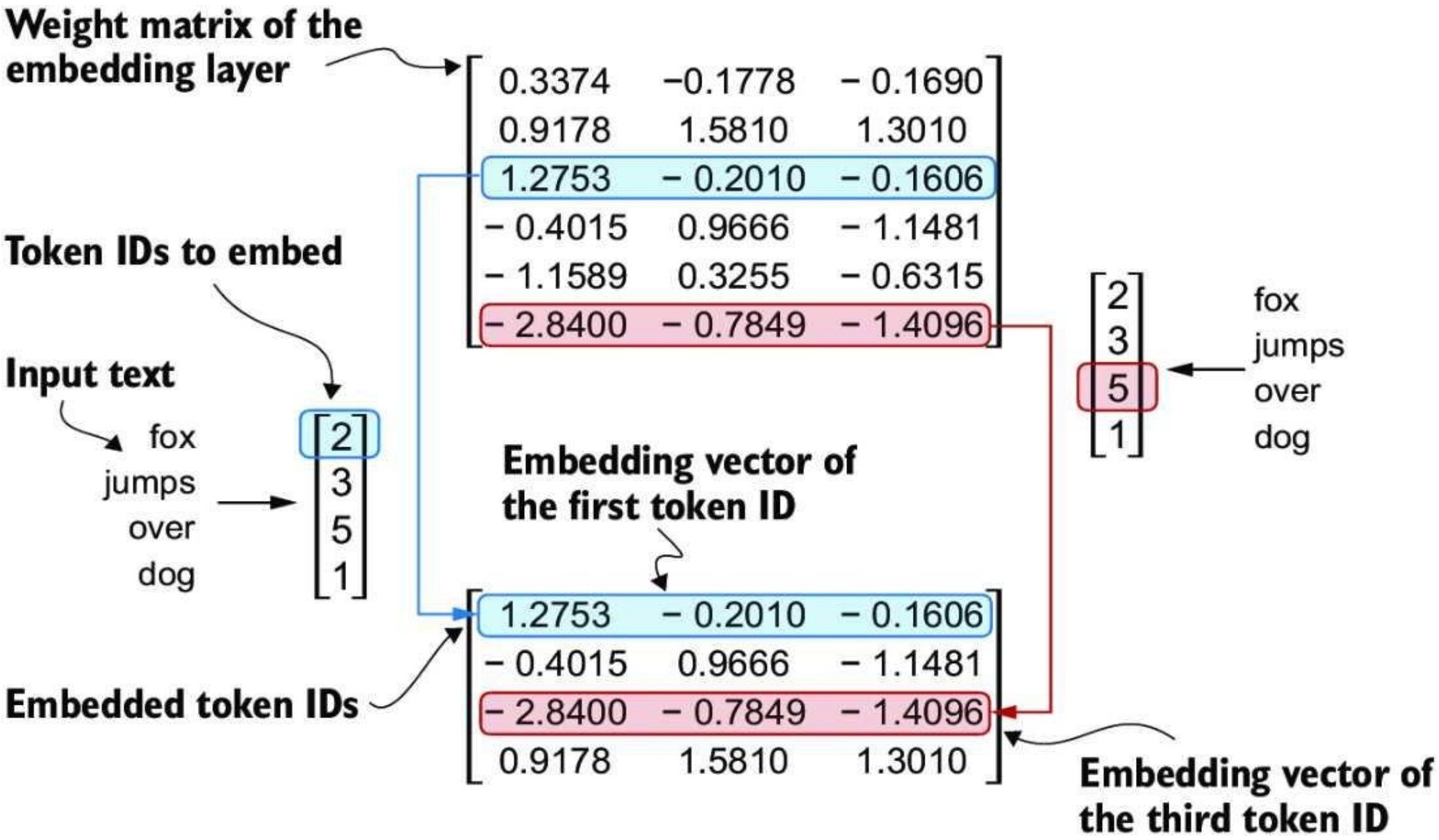
- you may know that small batch sizes require less memory during training but lead to more noisy model updates.
- Just like in regular deep learning, the batch size is a tradeoff and a hyperparameter to experiment with when training LLMs.(➔)

Creating token embeddings

- The **last step in preparing the input text** for LLM training is to convert the token IDs **into embedding vectors**



- As a preliminary step, we must **initialize these embedding weights with random values.**
- This **initialization serves as the starting point** for the LLM's learning process
- A continuous vector representation, or embedding, is necessary since GPT-like LLMs are deep neural networks trained with the backpropagation algorithm
- Let's see how the token ID to embedding vector conversion works with a hands-on example. Suppose we have the following four input tokens with IDs 2, 3, 5, and 1: (➔)
- The weight matrix of the **embedding layer contains small, random values**. **These values are optimized during LLM training as part of the LLM optimization itself**

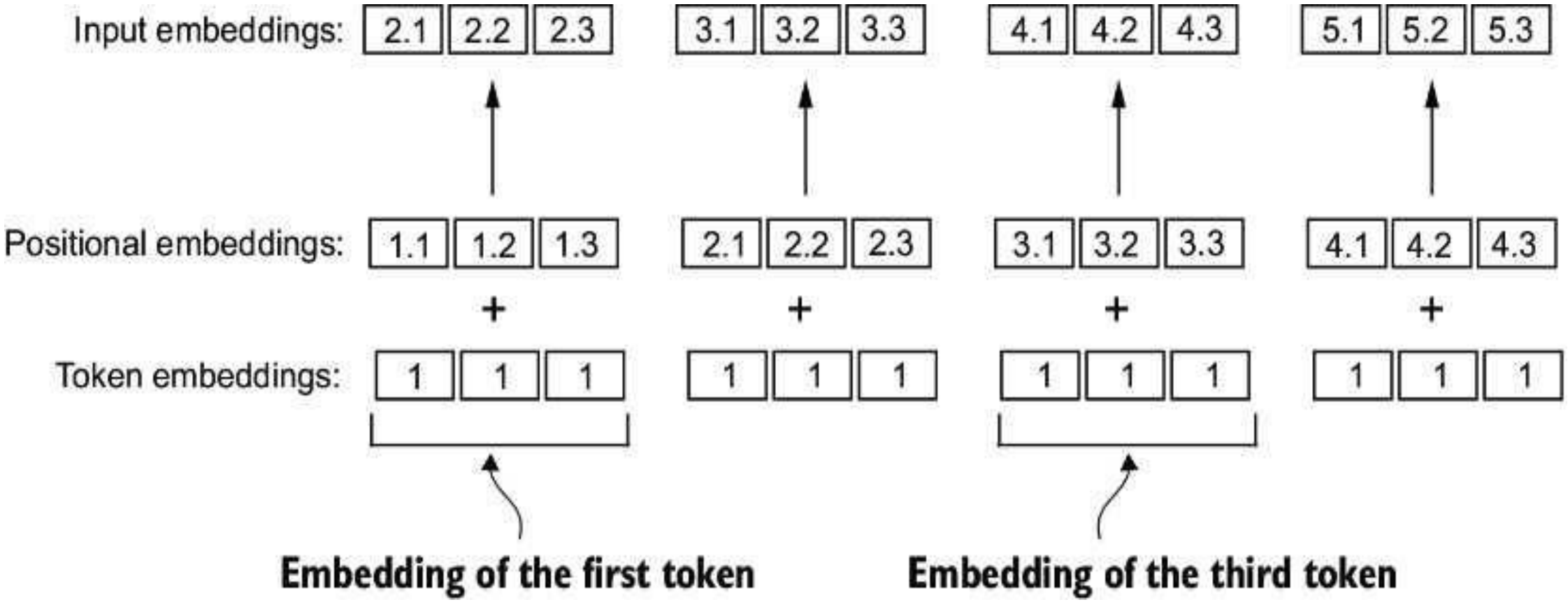

Weight matrix of the embedding layer
0.3374 −0.1778 − 0.1690
0.9178 1.5810 1.3010
1.2753 − 0.2010 − 0.1606
− 0.4015 0.9666 − 1.1481
− 1.1589 0.3255 − 0.6315
− 2.8400 − 0.7849 − 1.4096
Token IDs to embed
Input text
fox
jumps
over
dog
2
3
5
1
Embedding vector of the first token ID
1.2753 − 0.2010 − 0.1606
− 0.4015 0.9666 − 1.1481
− 2.8400 − 0.7849 − 1.4096
0.9178 1.5810 1.3010
Embedded token IDs
2
3
5
1
fox
jumps
over
dog
Embedding vector of the third token ID

- Embedding layers perform a lookup operation, retrieving the embedding vector corresponding to the token ID from the embedding layer's weight matrix.
- For instance, the embedding vector of the token ID 5 is the sixth row of the embedding layer weight matrix (it is the sixth instead of the fifth row because Python starts counting at 0). We assume that the token IDs were produced by the small vocabulary

Encoding word positions

- token embeddings are a suitable input for an LLM. However, a minor shortcoming of LLMs is that their self-attention mechanism doesn't have a notion of position or order for the tokens within a sequence.
- The way the previously introduced embedding layer works is that the same token ID always gets mapped to the same vector representation, regardless of where the token ID is positioned in the input sequence
- In principle, the deterministic, position-independent embedding of the token ID is good for reproducibility purposes.
- since the self-attention mechanism of LLMs itself is also position-agnostic, it is helpful to inject additional position information into the LLM.

- To achieve this, we can use two broad categories of position- aware embeddings: **relative positional embeddings** and **absolute positional embeddings.**
- Absolute positional embeddings are directly associated with specific positions in a sequence.
- For each position in the input sequence, a unique embedding is added to the token's embedding to convey its exact location.


Input embeddings:
2.1 2.2 2.3
3.1 3.2 3.3
4.1 4.2 4.3
5.1 5.2 5.3
Positional embeddings:
1.1 1.2 1.3
2.1 2.2 2.3
3.1 3.2 3.3
4.1 4.2 4.3
+
Token embeddings:
1 1 1
Embedding of the first token
Embedding of the third token

- Instead of focusing on the absolute position of a token, the emphasis of relative positional embeddings is on the relative position or distance between tokens
- This means the model learns the relationships in terms of “how far apart” rather than “at which exact position.”
- The advantage here is that the model can generalize better to sequences of varying lengths, even if it hasn't seen such lengths during training
- Both types of positional embeddings aim to augment the capacity of LLMs to understand the order and relationships between tokens, ensuring more accurate and context-aware predictions.
- The choice between them often depends on the specific application and the nature of the data being processed.

- OpenAI's GPT models use absolute positional embeddings that are optimized during the training process rather than being fixed or predefined like the positional encodings in the original transformer model.
- This optimization process is part of the model training itself. For now, let's create the initial positional embeddings to create the LLM inputs.
- Now, let's consider more realistic and useful embedding sizes and encode the input tokens into a 256- dimensional vector representation, which is smaller than what the original GPT-3 model used (in GPT-3, the embedding size is 12,288 dimensions) but still reasonable for experimentation.(➔)

- END of Unit-1